

LAPORAN PRAKTIKUM
POSTTEST (7)
ALGORITMA PEMROGRAMAN LANJUT

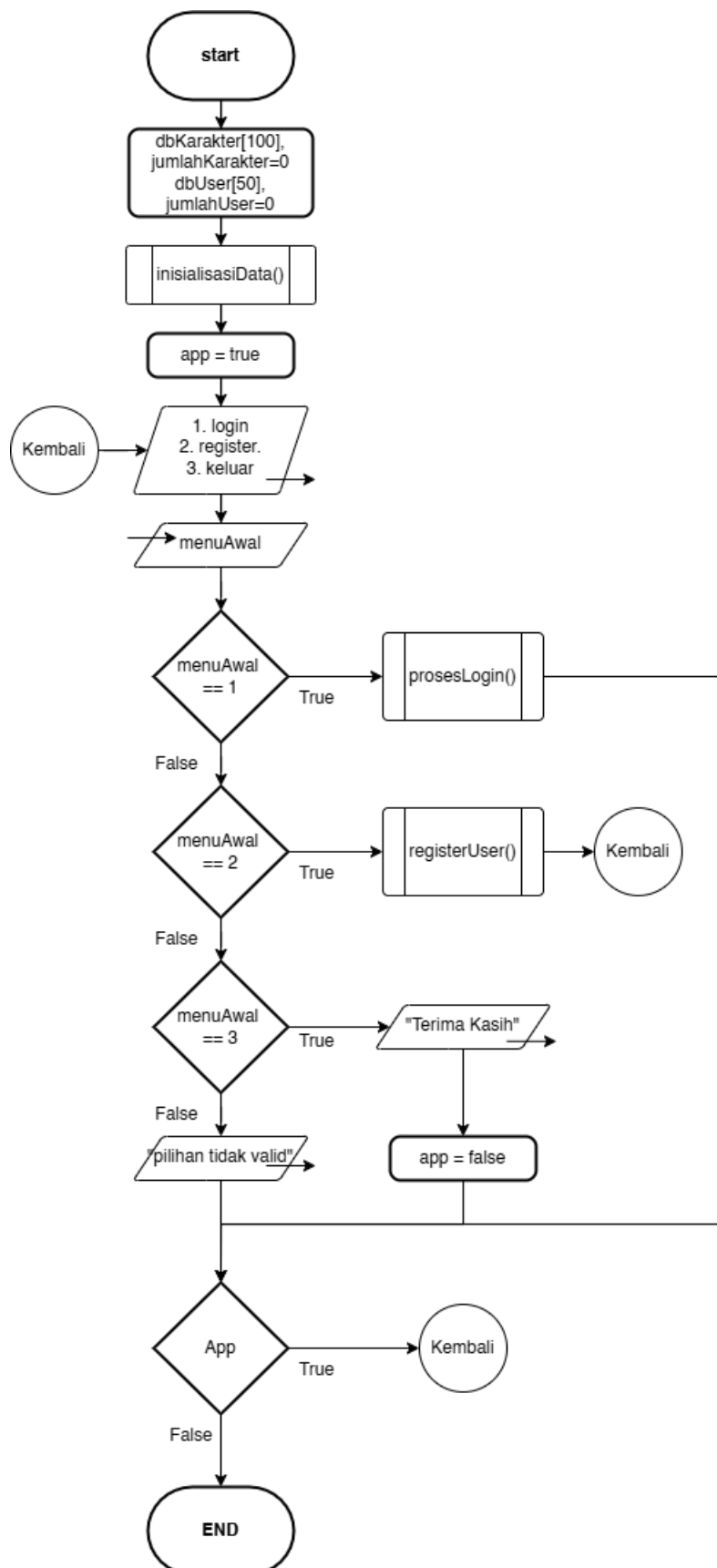


Disusun oleh:
Ajis Aditya Al Zahril (2509106048)
Kelas (B1 '25)

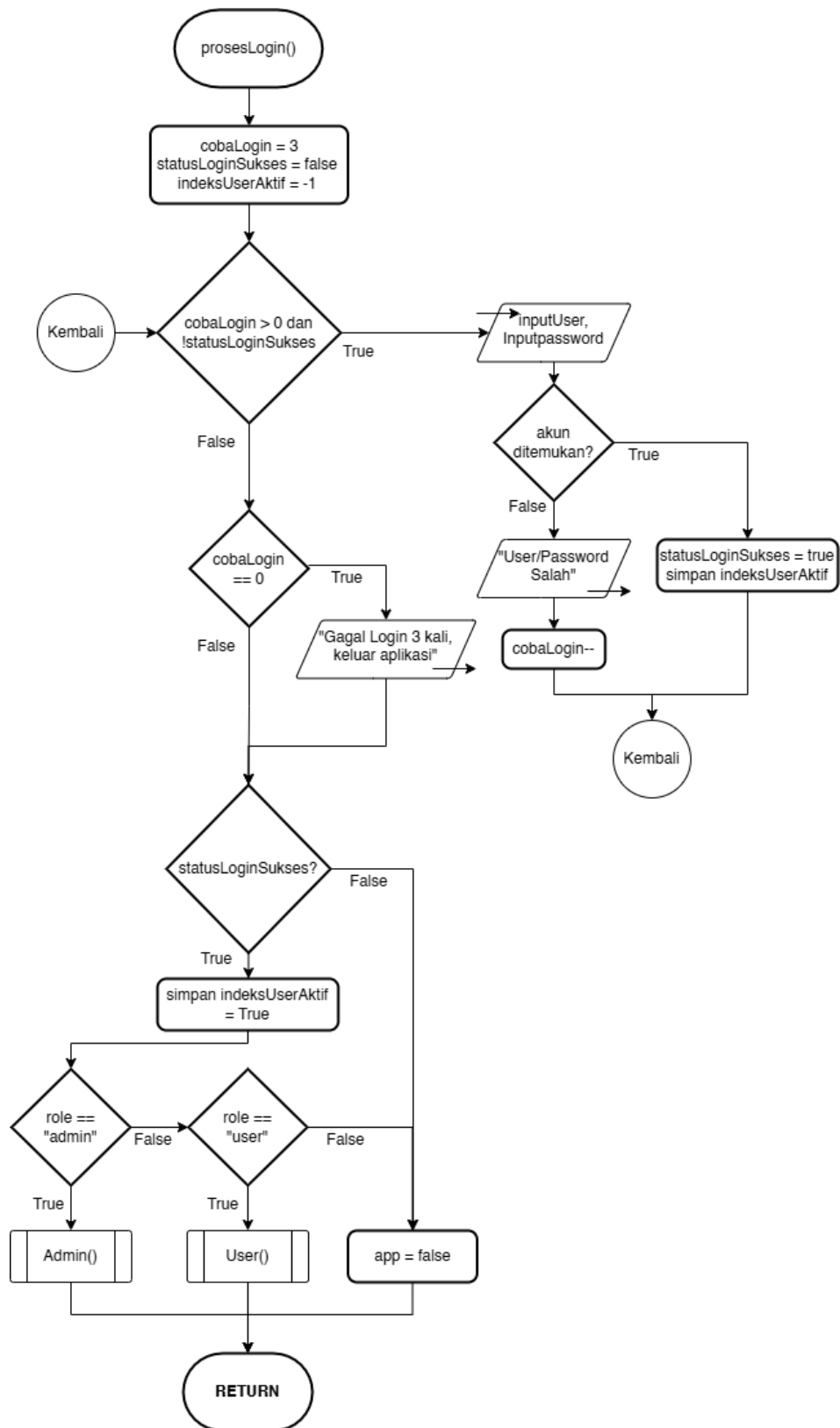
PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2026

1. Flowchart

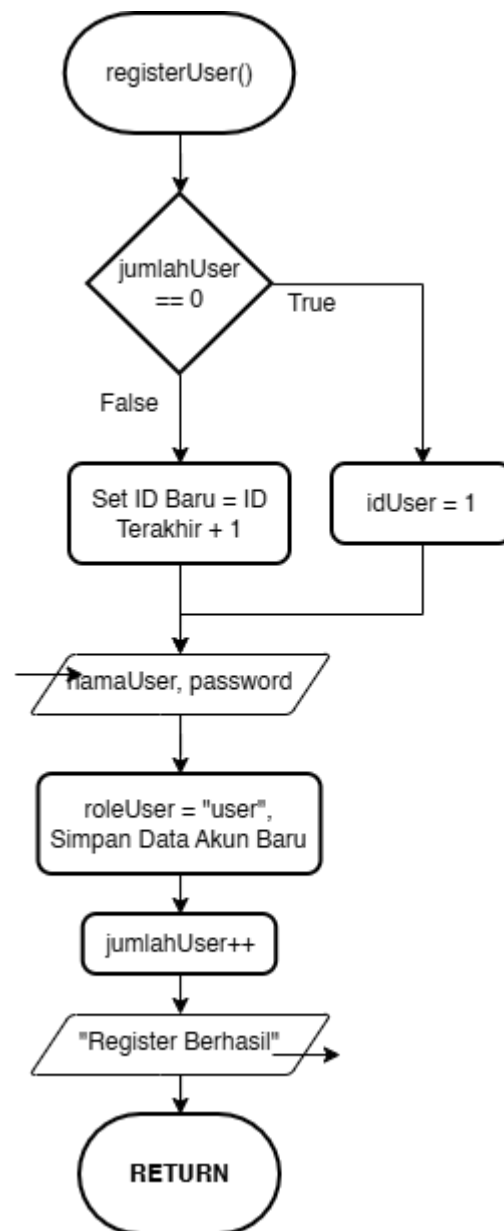
1. MAIN



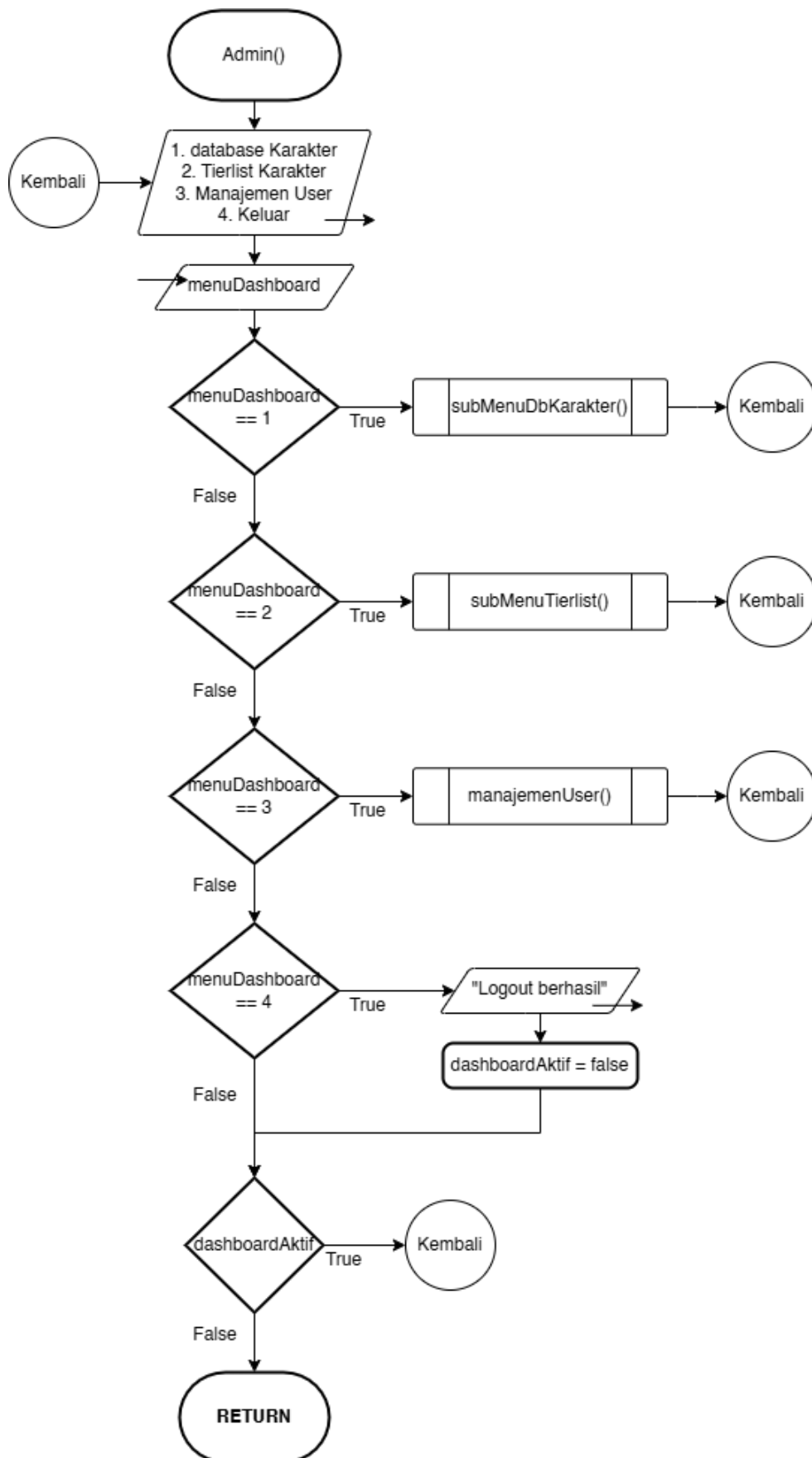
2. prosesLogin



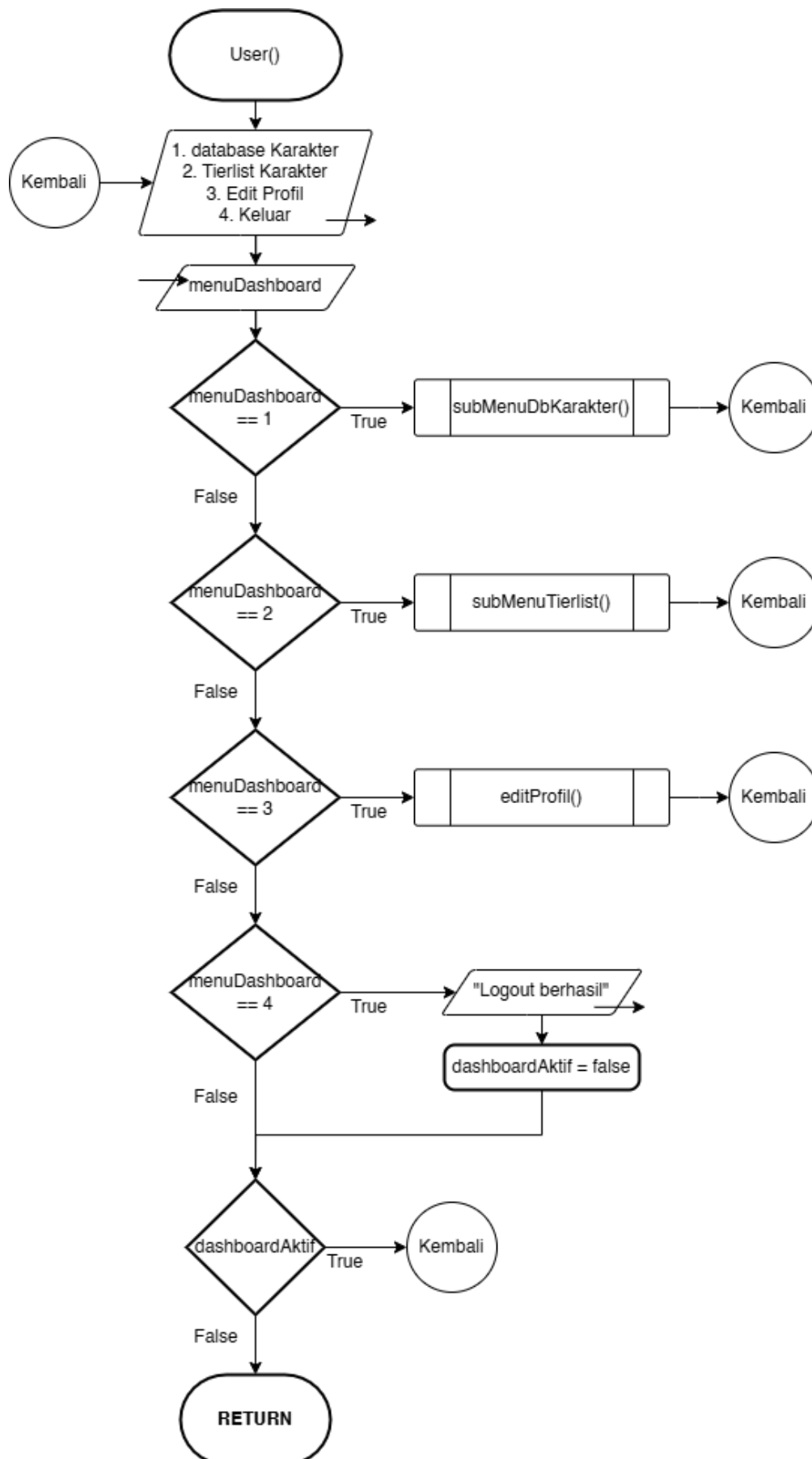
3. registerUser



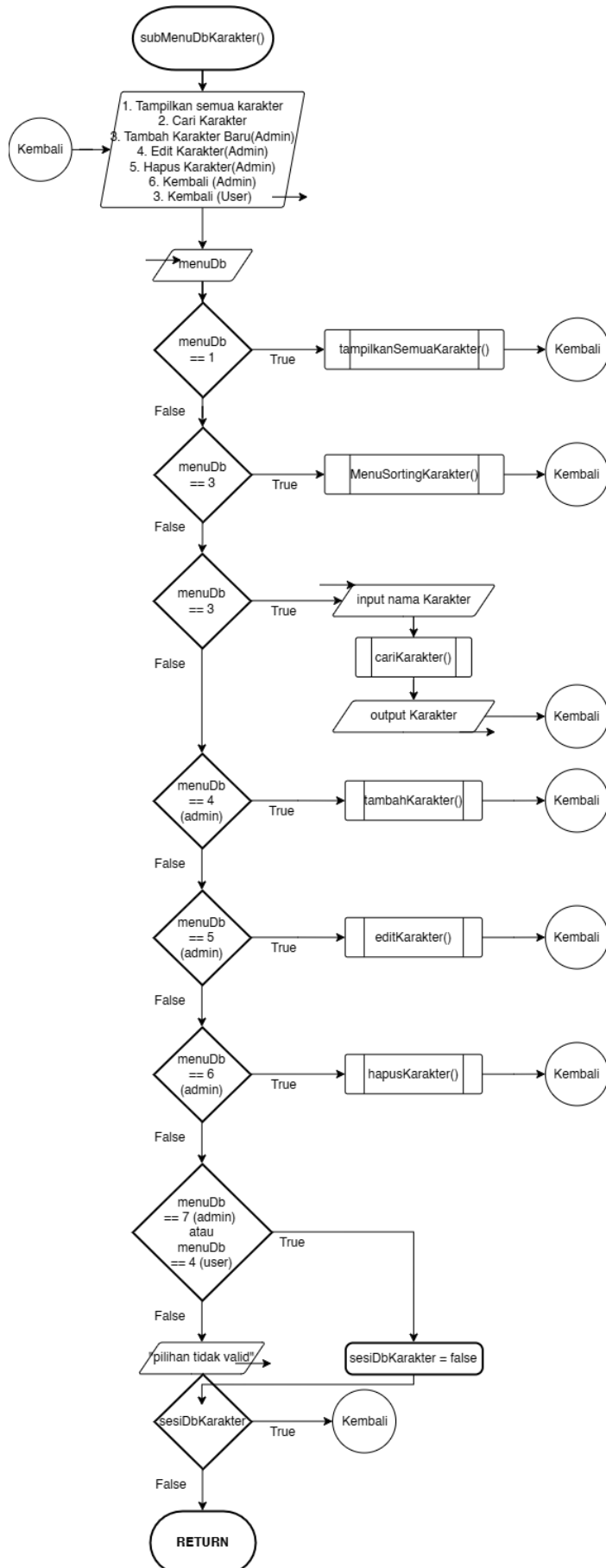
4. Admin



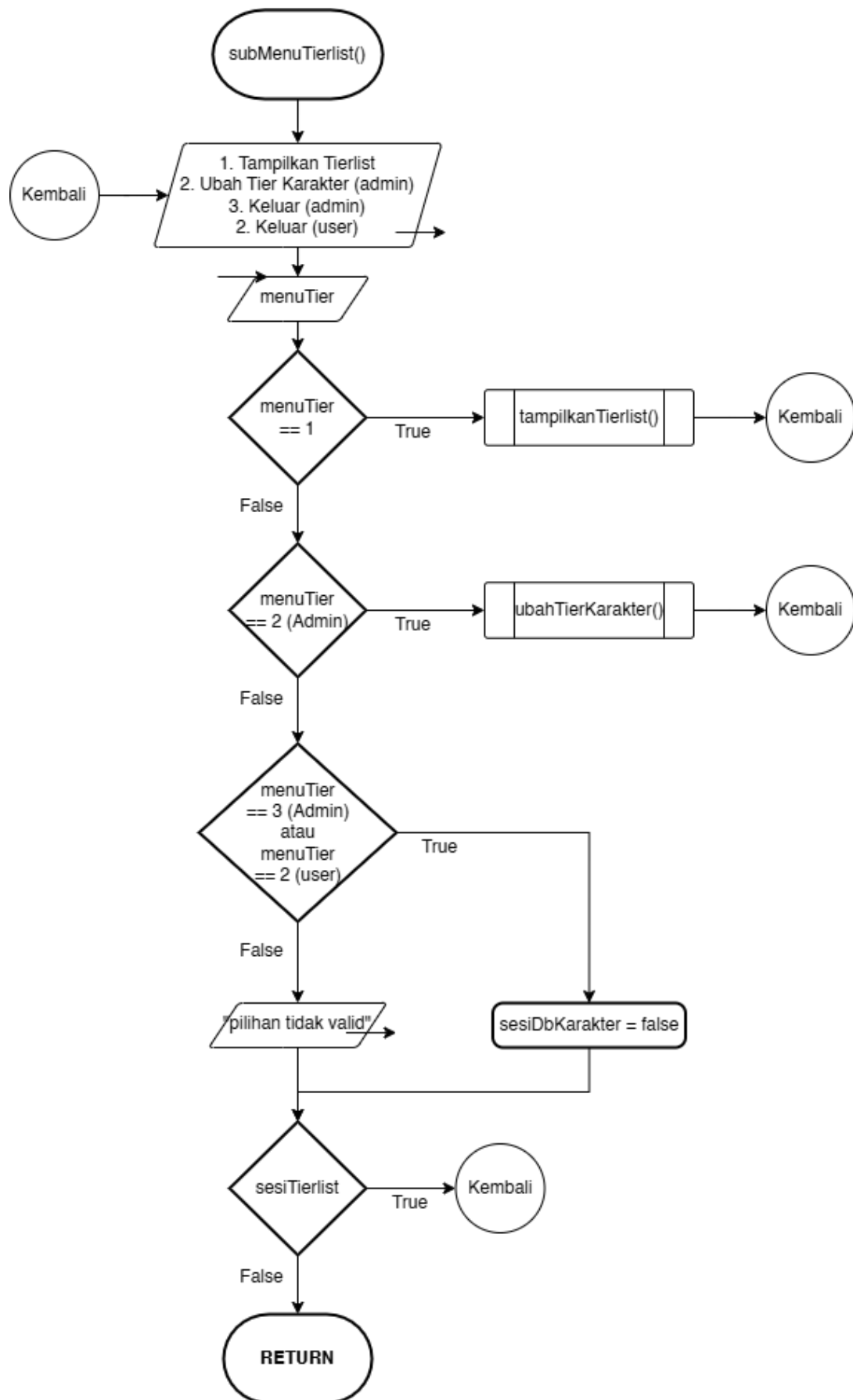
5. User



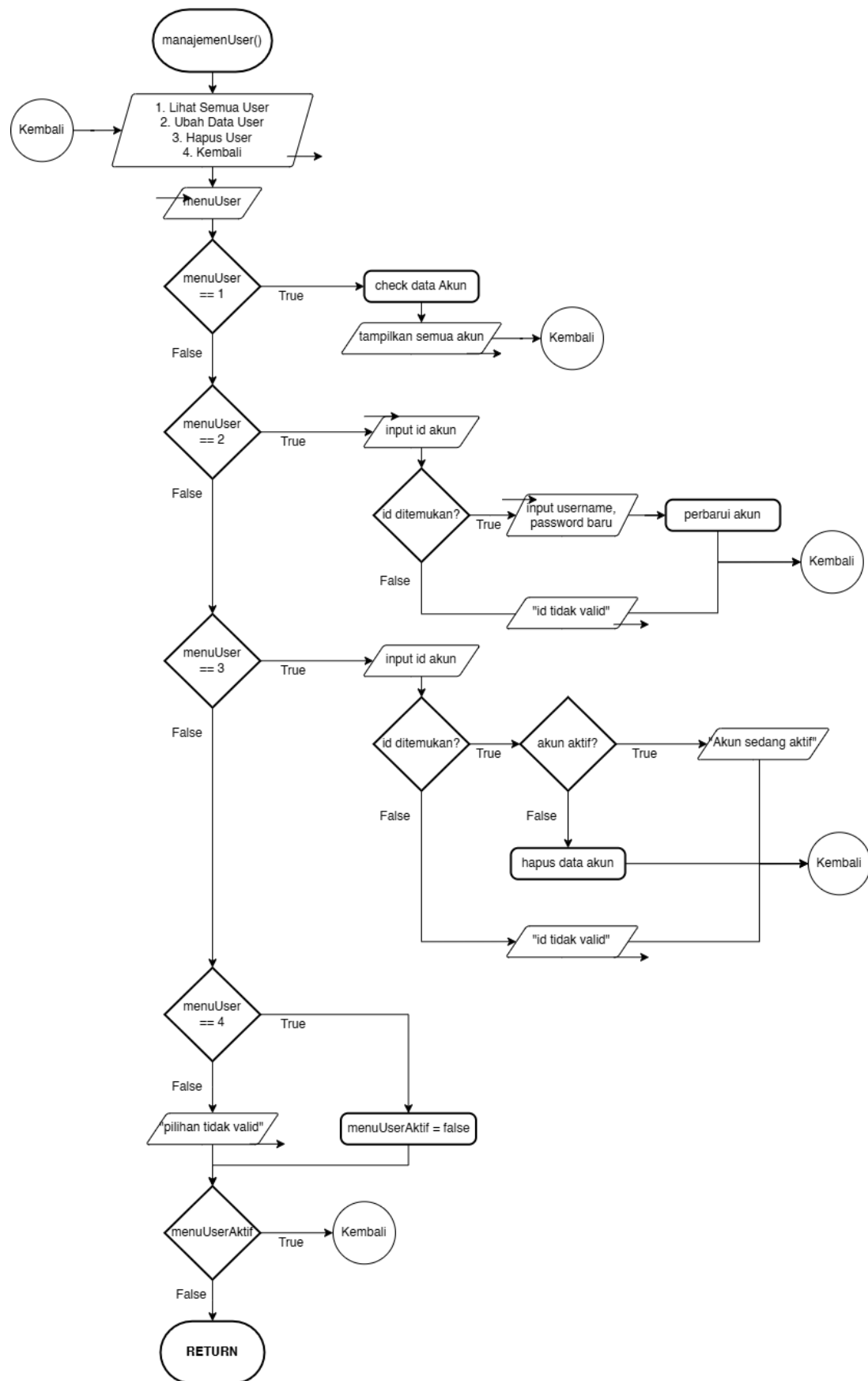
6. subMenuDbKarakter



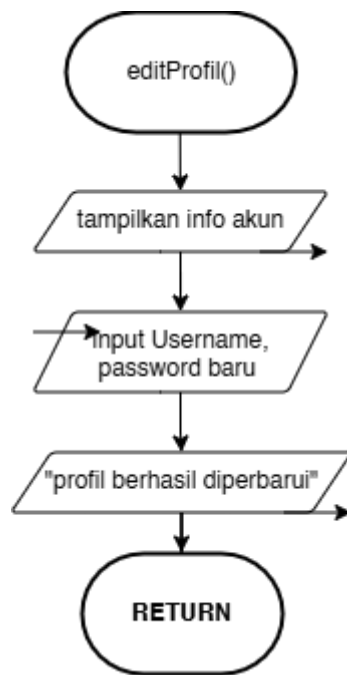
7. subMenuTierlist



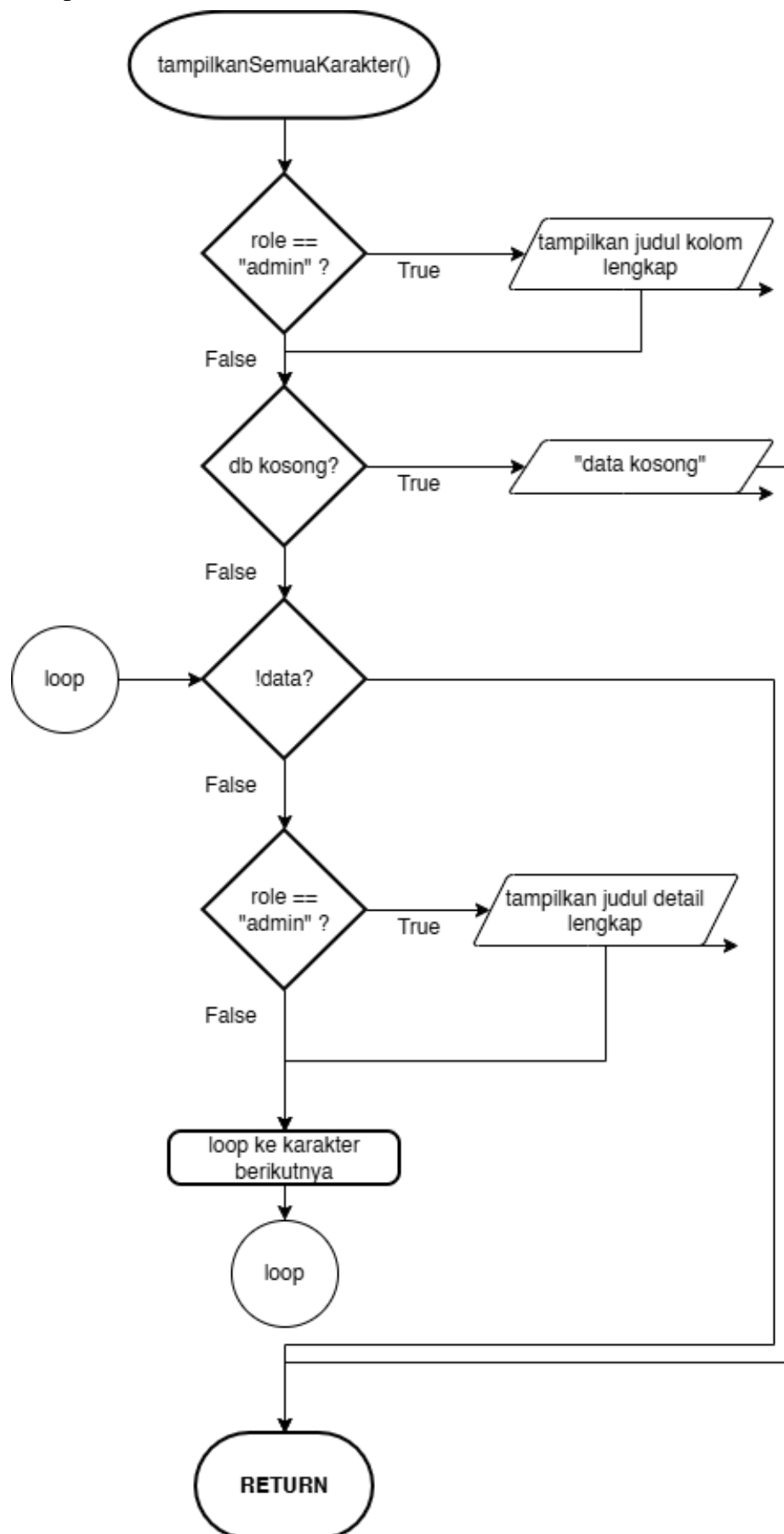
7. manajemenUser (Admin)



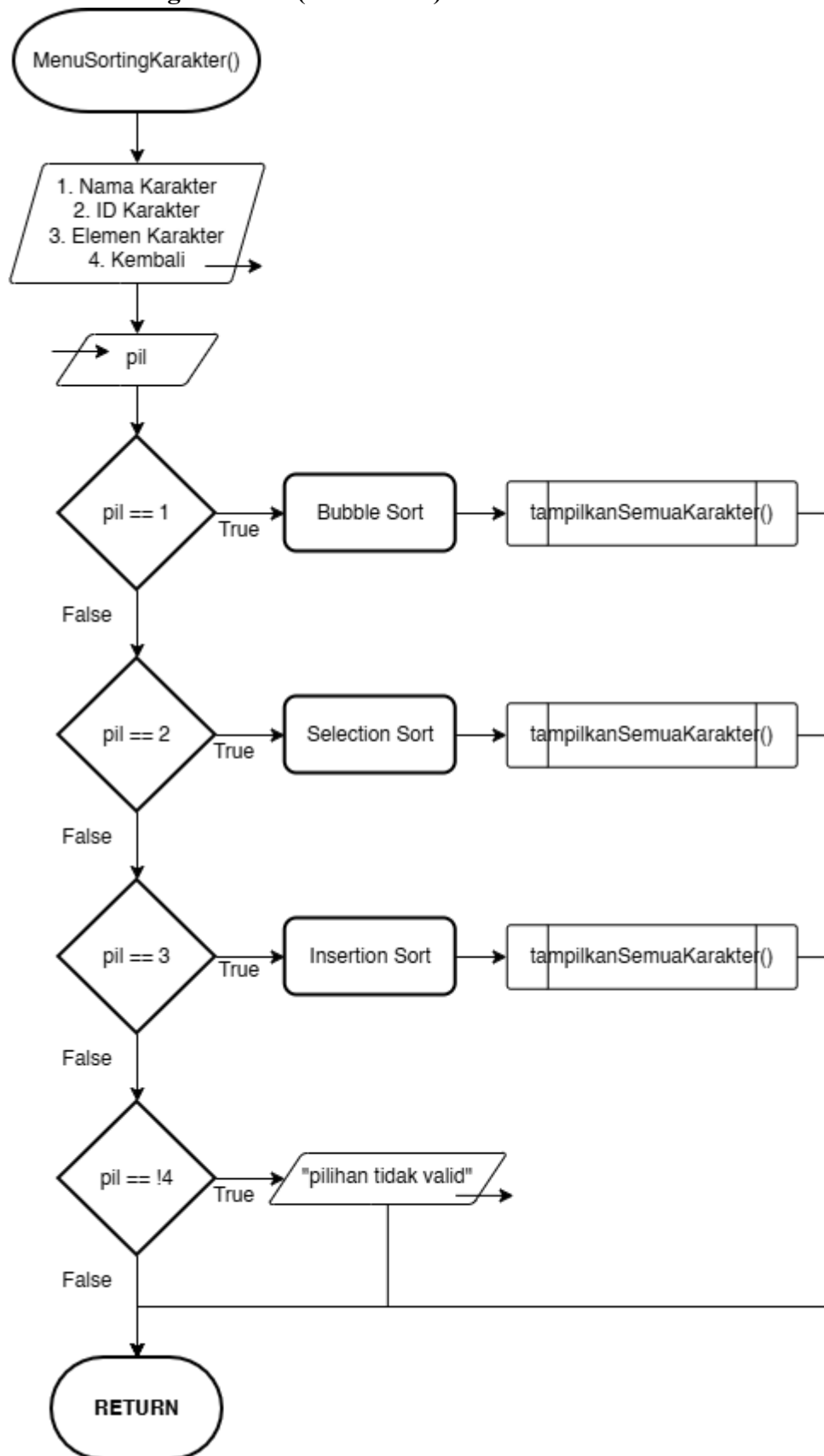
8. editProfil (User)



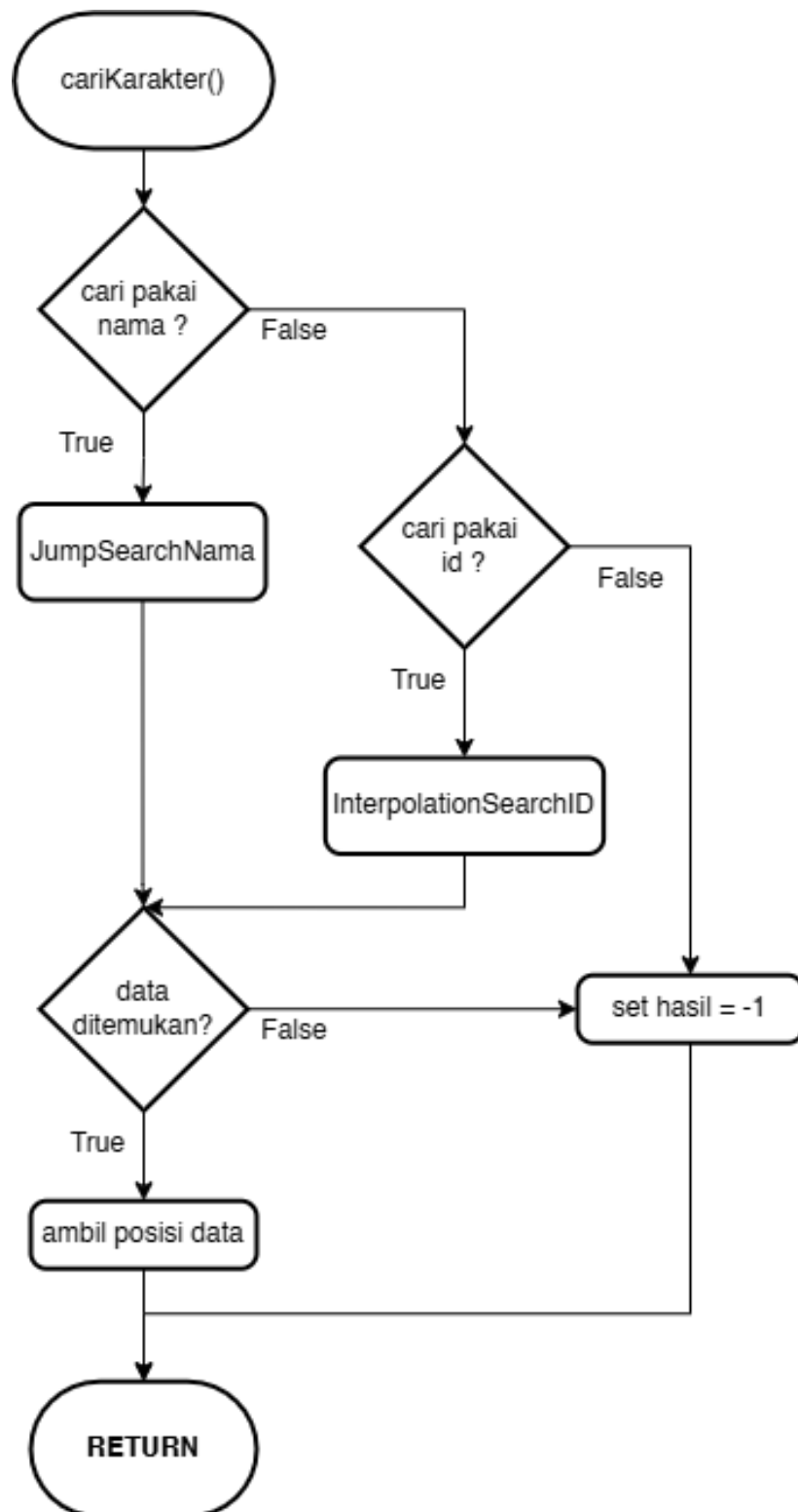
9. tampilkanSemuaKarakter



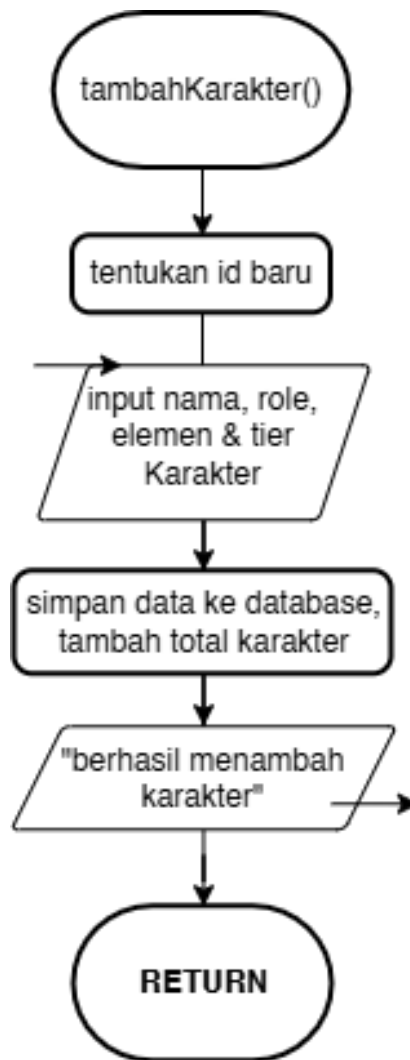
10. menuSortingKarakter (menu baru)



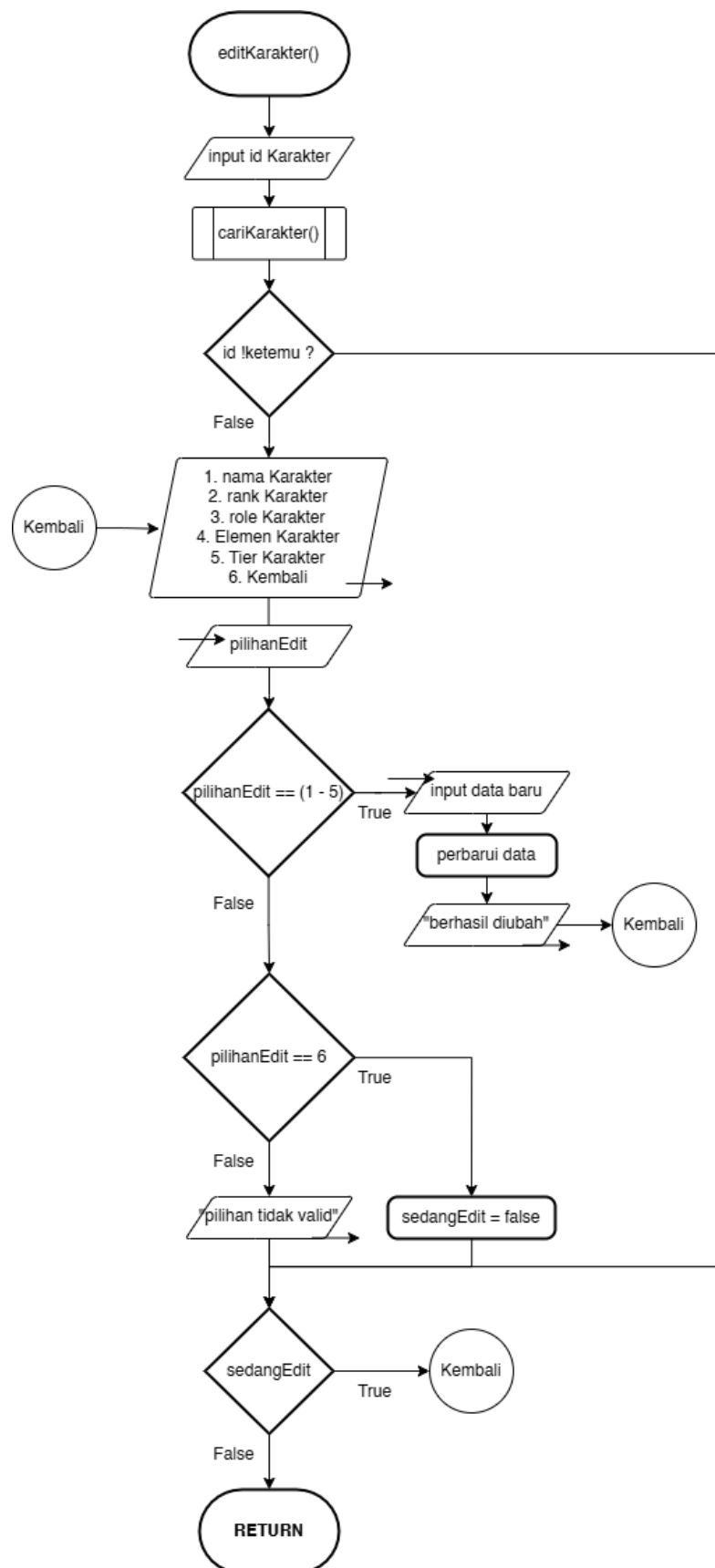
11. cariKarakter



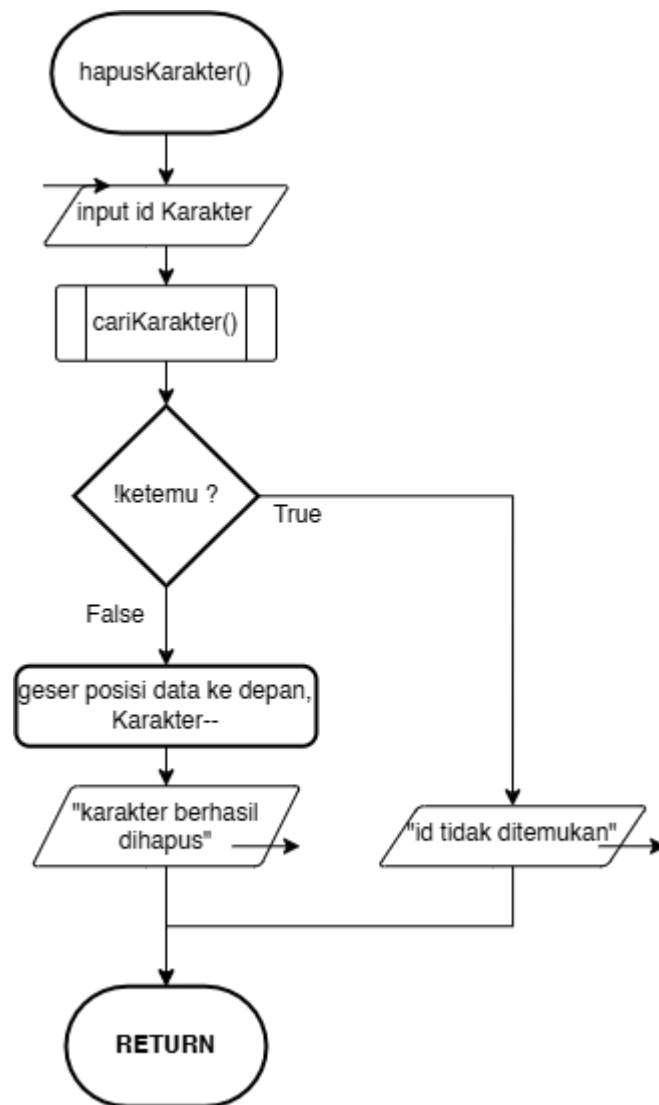
12. tambahKarakter



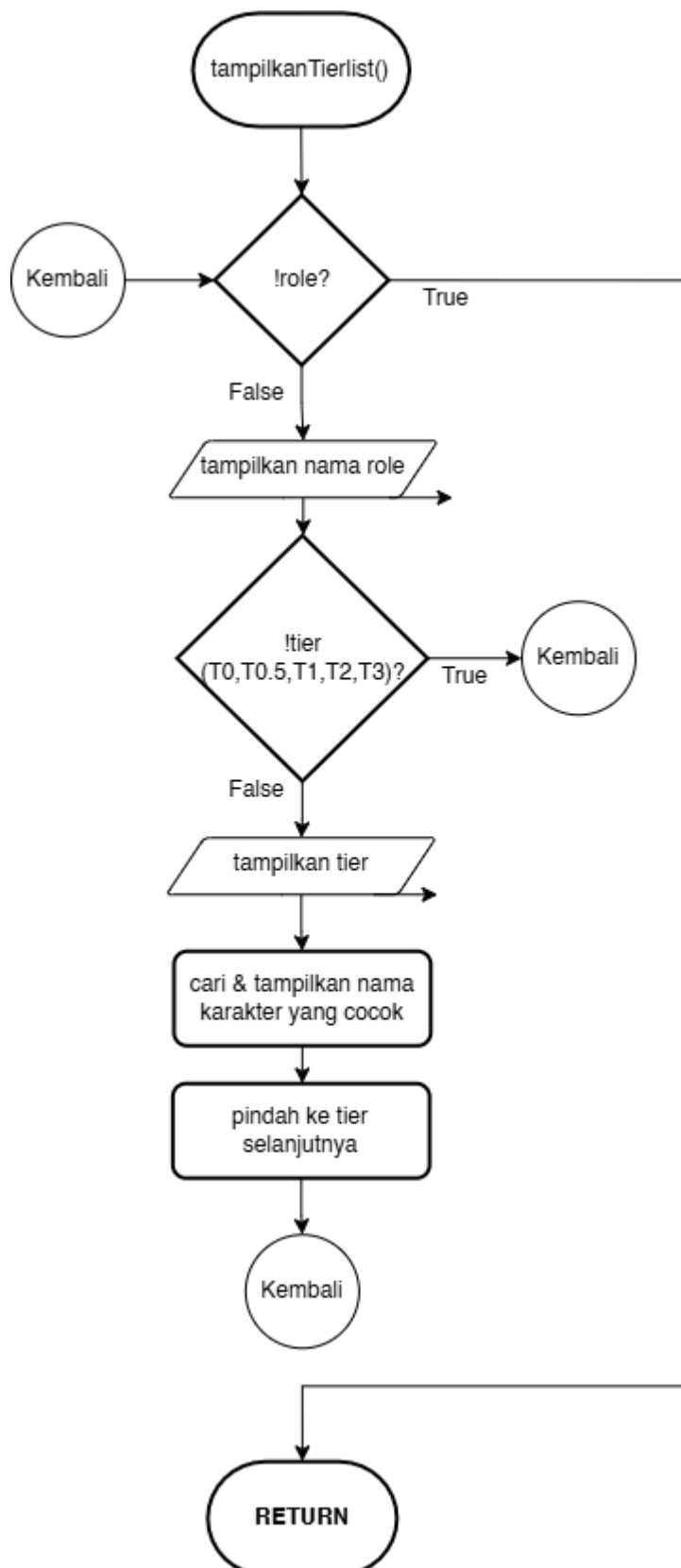
13. editKarakter



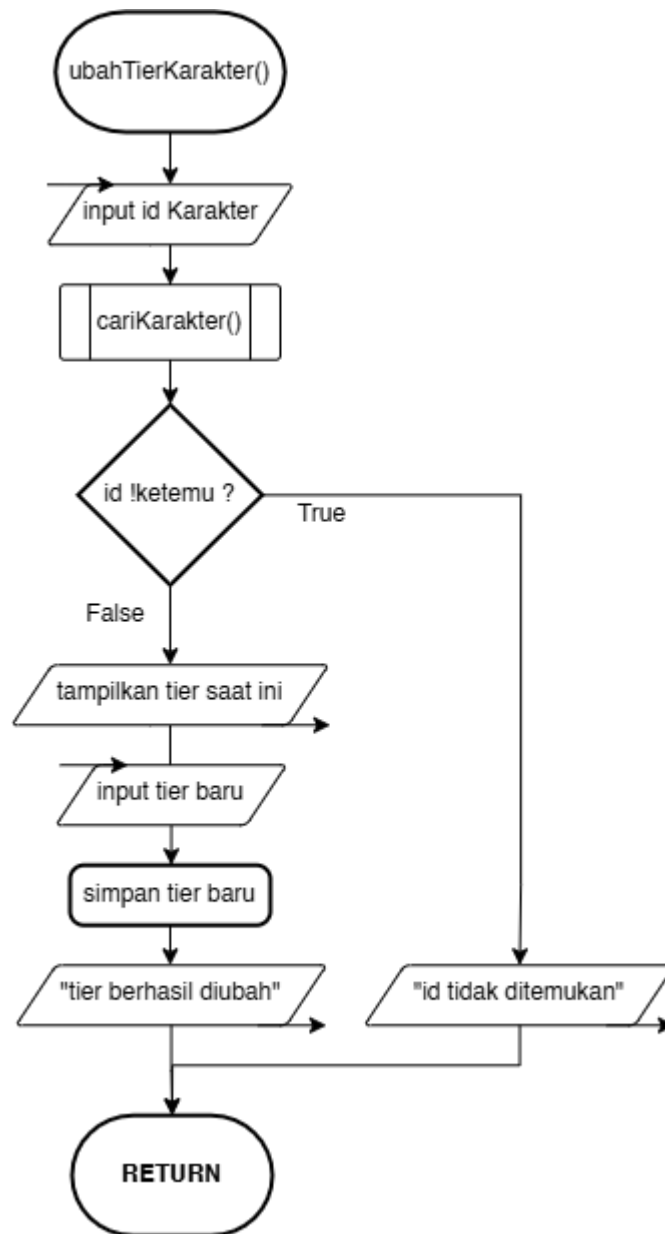
14. hapusKarakter



15. tampilkanTierlist



16. ubahTierKarakter



A. Terminator Session

- START Titik awal eksekusi program.
- END Program berhenti dan keluar dari memori.

B. Auth Session (Login & Register)

- Inisialisasi Sistem mengisi data awal (49 karakter & 3 akun) ke dalam array. Set App = true.
- Loop Utama Program terus berjalan selama App bernilai *true*.
- Input Menu Awal
 - 1 (Login): Diberikan 3 kali kesempatan. Input username & password. Jika cocok, masuk ke Dashboard sesuai *Role* (Admin/User). Jika gagal 3 kali, program berhenti (App = false).
 - 2 (Register): Input akun baru. ID dibuat otomatis (ID terakhir + 1), *Role* otomatis jadi "user".
 - 3 (Keluar): Set App = false untuk menghentikan program.
 - Else: Output "Pilihan tidak valid".

C. Dashboard Admin Session

- Memasuki Kondisi loop dashboardAktif == true.
- Pengecekan Menu:
 - 1 (Database Karakter): Membuka sub-menu olah data karakter.
 - 2 (Tierlist Karakter): Membuka sub-menu pengaturan tier.
 - 3 (Manajemen User): Membuka sub-menu pengaturan akun pengguna.
 - 4 (Logout): Set dashboardAktif = false untuk kembali ke menu login.

D. Dashboard User Session

- Memasuki Kondisi loop dashboardAktif == true.
- Pengecekan Menu:
 - 1 (Database Karakter): Hanya bisa melihat dan mencari data karakter.
 - 2 (Tierlist Karakter): Hanya bisa melihat daftar tierlist.
 - 3 (Edit Profil): Input Nama & Password baru untuk akun sendiri.
 - 4 (Logout): Set dashboardAktif = false untuk kembali ke menu login.

E. Menu DB Karakter Session

- Loop Sub-Menu Berjalan selama sesiDbKarakter == true.
- Aksi Menu:
 - 1 (Tampilkan Semua): Admin melihat data lengkap (ID/Tier), User melihat data terbatas.
 - 2 (Tampilkan menu Sorting): Admin & User bisa melihat seluruh data dengan metode sorting yang sudah ada.
 - 3 (Cari Karakter): Mencari berdasarkan nama menggunakan fungsi Rekursif. Jika ketemu, tampilkan detail data.
 - 4 (Admin - Tambah): Input data baru jika kapasitas < 100.
 - 5 (Admin - Edit): Cari ID, jika ketemu maka masuk ke loop perubahan data (Nama/Rank/Role/Elemen/Tier).
 - 6 (Admin - Hapus): Cari ID, jika ketemu maka geser posisi array ke depan dan kurangi jumlah total data.
 - Kembali (7 Admin / 4 User): Tutup sesi menu database.

F. Menu Tierlist Session

- Loop Sub-Menu Berjalan selama sesiTierlist == true.
- Aksi Menu:
 - 1 (Lihat Tierlist): Sistem melakukan perulangan bersarang (*nested loop*) untuk mengelompokkan data berdasarkan *Role* (Attack, Stun, dll) lalu berdasarkan *Tier* (T0-T3).
 - 2 (Admin - Ubah Tier): Cari ID karakter, tampilkan tier lama, input pilihan tier baru (T0-T3).
 - Kembali (3 Admin / 2 User): Tutup sesi menu tierlist.

G. Manajemen User Session

- Loop Sub-Menu Berjalan selama menuUserAktif == true.
- Aksi Menu:
 - 1 (Lihat Semua User): Menampilkan tabel berisi ID, Username, dan Role seluruh user.
 - 2 (Ubah Data User): Cari ID user, input Username & Password baru.
 - 3 (Hapus User): Cari ID. Cek jika ID == ID yang sedang login, maka ditolak. Jika tidak, hapus data dengan menggeser array.
 - 4 (Kembali): Tutup sesi manajemen user.

F. Menu Sorting Karakter Session (Menu baru)

- **Kondisi Akses:** Dipanggil saat User atau Admin memilih opsi 2 (Urutkan Karakter) pada Menu Database Karakter.
- **Aksi Menu:**
 - **1 (Nama Karakter Z-A / Descending):** Sistem mengurutkan data nama karakter dari huruf Z ke A menggunakan metode algoritma *Bubble Sort* (membandingkan dan menukar elemen yang bersebelahan). Setelahurut, sistem otomatis menampilkan tabel hasil.
 - **2 (ID Karakter Kecil-Besar / Ascending):** Sistem mengurutkan data ID karakter dari angka terkecil ke terbesar menggunakan metode algoritma *Selection Sort* (mencari nilai terkecil dan menempatkannya di posisi awal). Setelahurut, sistem otomatis menampilkan tabel hasil.
 - **3 (Elemen Karakter A-Z / Ascending):** Sistem mengurutkan data elemen karakter (Ice, Fire, dll) sesuai abjad dari A ke Z menggunakan algoritma *Insertion Sort* (menyisipkan elemen ke posisi yang tepat). Setelahurut, sistem otomatis menampilkan tabel hasil.
 - **4 (Batal):** Membatalkan instruksi sorting dan sistem akan kembali ke Menu Database Karakter tanpa mengubah urutan data saat ini.

2. Deskripsi Singkat Program

Program ini adalah aplikasi Database dan Tierlist Karakter game "Zenless Zone Zero" yang berfungsi untuk mengelola dan menampilkan susunan data karakter. Program ini dilengkapi dengan sistem autentikasi (Login dan Register), di mana pengguna diberikan maksimal 3 kali kesempatan untuk login. Jika login berhasil, sistem akan mendeteksi hak akses (*role*) pengguna untuk membedakan tampilan menu utama antara Admin dan User biasa.

Admin memiliki hak akses penuh untuk melakukan operasi CRUD (menambah, membaca, mengedit, dan menghapus) pada data karakter dan mengubah susunan *tierlist*, sedangkan User hanya diberi batasan untuk mencari, membaca data, dan mengedit profil akunnya sendiri. Program akan terus menampilkan menu secara berulang (*looping*) hingga pengguna memilih opsi untuk keluar.

Program ini juga mencakup fitur seperti sorting untuk efisiensi pencarian data secara cepat dan tepat.

Program ini mencakup beberapa variabel, yaitu:

- dbKarakter (Array of Struct) : menyimpan kumpulan data detail dari setiap karakter (seperti ID, nama, elemen, role, rank, dan tier).
- dbUser (Array of Struct) : menyimpan kumpulan data akun pengguna beserta hak aksesnya (Admin/User).
- cobaLogin (int) : menyimpan batas maksimal kesempatan login pengguna (3).
- aplikasiBerjalan (bool) : indikator utama untuk menjalankan atau menghentikan *looping* program secara keseluruhan (true/false).
- statusLoginSukses (bool) : indikator penentu keberhasilan proses verifikasi nama dan password akun (true/false).
- indeksUserAktif (int) : menyimpan posisi laci (indeks array) dari akun yang sedang login untuk melacak data dan hak akses akun tersebut.
- keywordPencarian (string) : menyimpan input kata kunci dari pengguna saat menggunakan fitur pencarian nama karakter yang menggunakan Interpolation Search dan Jump search.
- Penggunaan User defined library dan berbagai penggunaan error handling agar lebih efisien.

3. Source Code

Source Code:

A. .CPP

```
#include <iostream>
#include <string>
#include <iomanip>
#include <cstdlib>
#include <cmath>
#include <algorithm>
#include <stdexcept>
#include "validasi.h"

using namespace std;

struct DetailKarakter {
    string roleKarakter;
    string elemenKarakter;
};

struct Karakter {
    int idKarakter;
    string namaKarakter;
    DetailKarakter detail;
    string rankKarakter;
    string tierKarakter;
};

struct AkunUser {
    int idUser;
    string namaUser;
    string password;
    string roleUser;
};

void clearScreen() {
    #ifdef _WIN32
        system("cls");
    #else
        system("clear");
    #endif
}

void inisialisasiData(Karakter dbKarakter[], int &jumlahKarakter, AkunUser
dbUser[], int &jumlahUser) {
    dbKarakter[jumlahKarakter++] = {1, "Alice", {"Anomaly", "Physical"}, "S",
    "T0.5"};
    dbKarakter[jumlahKarakter++] = {2, "Anby Demara", {"Stun", "Electric"}, "A",
```



```

"T3";
    dbKarakter[jumlahKarakter++] = {3, "Anby: Soldier 0", {"Attack",
"Electric"}, "S", "T0.5"};
    dbKarakter[jumlahKarakter++] = {4, "Anton", {"Attack", "Electric"}, "A",
"T2"};
    dbKarakter[jumlahKarakter++] = {5, "Aria", {"Anomaly", "Ether"}, "S",
"T0.5"};
    dbKarakter[jumlahKarakter++] = {6, "Astra Yao", {"Support", "Ether"}, "S",
"T0"};
    dbKarakter[jumlahKarakter++] = {7, "Banyue", {"Rupture", "Fire"}, "S",
"T0.5"};
    dbKarakter[jumlahKarakter++] = {8, "Ben", {"Defense", "Fire"}, "S", "T3"};
    dbKarakter[jumlahKarakter++] = {9, "Billy", {"Attack", "Physical"}, "A",
"T2"};
    dbKarakter[jumlahKarakter++] = {10, "Burnice", {"Anomaly", "Fire"}, "S",
"T1"};
    dbKarakter[jumlahKarakter++] = {11, "Caesar", {"Defense", "Physical"}, "S",
"T2"};
    dbKarakter[jumlahKarakter++] = {12, "Corin", {"Attack", "Physical"}, "A",
"T2"};
    dbKarakter[jumlahKarakter++] = {13, "Dialyn", {"Stun", "Physical"}, "S",
"T0"};
    dbKarakter[jumlahKarakter++] = {14, "Ellen", {"Attack", "Ice"}, "S", "T1"};
    dbKarakter[jumlahKarakter++] = {15, "Evelyn", {"Attack", "Fire"}, "S",
"T0.5"};
    dbKarakter[jumlahKarakter++] = {16, "Grace", {"Anomaly", "Electric"}, "S",
"T2"};
    dbKarakter[jumlahKarakter++] = {17, "Harumasa", {"Attack", "Electric"}, "S",
"T1"};
    dbKarakter[jumlahKarakter++] = {18, "Hugo", {"Attack", "Ice"}, "S", "T1"};
    dbKarakter[jumlahKarakter++] = {19, "Jane Doe", {"Anomaly", "Physical"},
"S", "T1"};
    dbKarakter[jumlahKarakter++] = {20, "Ju Fufu", {"Stun", "Fire"}, "S",
"T0.5"};
    dbKarakter[jumlahKarakter++] = {21, "Koleda", {"Stun", "Fire"}, "S", "T2"};
    dbKarakter[jumlahKarakter++] = {22, "Lighter", {"Stun", "Fire"}, "S",
"T0.5"};
    dbKarakter[jumlahKarakter++] = {23, "Lucia", {"Support", "Ether"}, "S",
"T0"};
    dbKarakter[jumlahKarakter++] = {24, "Lucy", {"Support", "Fire"}, "A", "T2"};
    dbKarakter[jumlahKarakter++] = {25, "Lycaon", {"Stun", "Ice"}, "S", "T1"};
    dbKarakter[jumlahKarakter++] = {26, "Manato", {"Rupture", "Fire"}, "A",
"T1"};
    dbKarakter[jumlahKarakter++] = {27, "Miyabi", {"Anomaly", "Frost"}, "S",
"T0"};
    dbKarakter[jumlahKarakter++] = {28, "Nekomata", {"Attack", "Physical"}, "S",
"T2"};
    dbKarakter[jumlahKarakter++] = {29, "Nicole", {"Support", "Ether"}, "A",
"T0.5"};
    dbKarakter[jumlahKarakter++] = {30, "Orphie & Magus", {"Attack", "Fire"},
"S", "T0.5"};

```

```

        dbKarakter[jumlahKarakter++] = {31, "Pan Yinhu", {"Defense", "Physical"}, "A", "T1"};
        dbKarakter[jumlahKarakter++] = {32, "Piper", {"Anomaly", "Physical"}, "A", "T1"};
        dbKarakter[jumlahKarakter++] = {33, "Pulchra", {"Stun", "Physical"}, "A", "T2"};
        dbKarakter[jumlahKarakter++] = {34, "Qingyi", {"Stun", "Electric"}, "S", "T1"};
        dbKarakter[jumlahKarakter++] = {35, "Rina", {"Support", "Electric"}, "S", "T2"};
        dbKarakter[jumlahKarakter++] = {36, "Seed", {"Attack", "Electric"}, "S", "T0.5"};
        dbKarakter[jumlahKarakter++] = {37, "Seth", {"Defense", "Electric"}, "A", "T3"};
        dbKarakter[jumlahKarakter++] = {38, "Soldier 11", {"Attack", "Fire"}, "S", "T1"};
        dbKarakter[jumlahKarakter++] = {39, "Soukaku", {"Support", "Ice"}, "A", "T1"};
        dbKarakter[jumlahKarakter++] = {40, "Sunna", {"Support", "Physical"}, "S", "T0"};
        dbKarakter[jumlahKarakter++] = {41, "Trigger", {"Stun", "Electric"}, "S", "T0.5"};
        dbKarakter[jumlahKarakter++] = {42, "Vivian", {"Anomaly", "Ether"}, "S", "T0.5"};
        dbKarakter[jumlahKarakter++] = {43, "Yanagi", {"Anomaly", "Electric"}, "S", "T1"};
        dbKarakter[jumlahKarakter++] = {44, "Ye Shunguang", {"Attack", "Honed Edge"}, "S", "T0"};
        dbKarakter[jumlahKarakter++] = {45, "Yidhari", {"Rupture", "Ice"}, "S", "T0.5"};
        dbKarakter[jumlahKarakter++] = {46, "Yixuan", {"Rupture", "Auric Ink"}, "S", "T0"};
        dbKarakter[jumlahKarakter++] = {47, "Yuzuha", {"Support", "Physical"}, "S", "T0"};
        dbKarakter[jumlahKarakter++] = {48, "Zhao", {"Defense", "Ice"}, "S", "T0.5"};
        dbKarakter[jumlahKarakter++] = {49, "Zhu Yuan", {"Attack", "Ether"}, "S", "T1"};

        dbUser[jumlahUser++] = {1, "ajis", "048", "admin"};
        dbUser[jumlahUser++] = {2, "user", "123", "user"};
        dbUser[jumlahUser++] = {3, "wow", "123", "heker"};
    }

```

```

int interpolationSearchID(Karakter* dbKarakter, int jumlahKarakter, int* targetId) {
    for (int i = 0; i < jumlahKarakter - 1; i++)
        for (int j = 0; j < jumlahKarakter - i - 1; j++)
            if (dbKarakter[j].idKarakter > dbKarakter[j + 1].idKarakter)
                swap(dbKarakter[j], dbKarakter[j + 1]);
}

```

```

        int low = 0, high = jumlahKarakter - 1;
        while (low <= high && *targetId >= dbKarakter[low].idKarakter && *targetId
<= dbKarakter[high].idKarakter) {
            if (low == high) {
                if (dbKarakter[low].idKarakter == *targetId) return low;
                return -1;
            }
            int pos = low + (((double)(high - low) / (dbKarakter[high].idKarakter -
dbKarakter[low].idKarakter)) * (*targetId - dbKarakter[low].idKarakter));
            if (dbKarakter[pos].idKarakter == *targetId) return pos;
            if (dbKarakter[pos].idKarakter < *targetId) low = pos + 1;
            else high = pos - 1;
        }
        return -1;
    }

int jumpSearchNama(Karakter* dbKarakter, int jumlahKarakter, string* targetName)
{
    for (int i = 0; i < jumlahKarakter - 1; i++)
        for (int j = 0; j < jumlahKarakter - i - 1; j++)
            if (dbKarakter[j].namaKarakter > dbKarakter[j + 1].namaKarakter)
                swap(dbKarakter[j], dbKarakter[j + 1]);

    int step = sqrt(jumlahKarakter);
    int prev = 0;
    while (dbKarakter[min(step, jumlahKarakter) - 1].namaKarakter < *targetName)
    {
        prev = step;
        step += sqrt(jumlahKarakter);
        if (prev >= jumlahKarakter) return -1;
    }
    while (prev < min(step, jumlahKarakter) && dbKarakter[prev].namaKarakter <
*targetName)
        prev++;
    if (prev < jumlahKarakter && dbKarakter[prev].namaKarakter == *targetName)
return prev;
    return -1;
}

void tampilkanSemuaKarakter(Karakter dbKarakter[], int jumlahKarakter, string
roleSaatIni) {
    clearScreen();
    if (roleSaatIni == "admin") {
        cout << "\n" << left << setw(5) << "ID" << setw(20) << "Nama" <<
setw(10) << "Rank"
                << setw(15) << "Role" << setw(15) << "Elemen" << setw(10) << "Tier"
<< "\n";
        cout << "-----\n";
    } else {
        cout << "\n" << left << setw(20) << "Nama" << setw(10) << "Rank"

```

```

        << setw(15) << "Role" << setw(15) << "Elemen" << "\n";
        cout << "-----
\n";
    }
    if (jumlahKarakter == 0) cout << "Data karakter kosong.\n";
    for (int i = 0; i < jumlahKarakter; i++) {
        if (roleSaatIni == "admin")
            cout << left << setw(5) << dbKarakter[i].idKarakter << setw(20) <<
dbKarakter[i].namaKarakter
                << setw(10) << dbKarakter[i].rankKarakter << setw(15) <<
dbKarakter[i].detail.roleKarakter
                << setw(15) << dbKarakter[i].detail.elemenKarakter << setw(10)
<< dbKarakter[i].tierKarakter << "\n";
        else
            cout << left << setw(20) << dbKarakter[i].namaKarakter << setw(10)
<< dbKarakter[i].rankKarakter
                << setw(15) << dbKarakter[i].detail.roleKarakter << setw(15) <<
dbKarakter[i].detail.elemenKarakter << "\n";
    }
}

void menuSortingKarakter(Karakter dbKarakter[], int jumlahKarakter, string
roleSaatIni) {
    clearScreen();
    cout << "\n= URUTKAN KARAKTER =\n";
    cout << "1. Nama Karakter - Bubble Sort\n";
    cout << "2. ID Karakter - Selection Sort\n";
    cout << "3. Elemen Karakter - Insertion Sort\n";
    cout << "4. Kembali\n";

    try {
        int pil = inputInt("Pilih metode sorting: ");
        validasiMenu(pil, 1, 4);

        if (pil == 1) {
            for (int i = 0; i < jumlahKarakter - 1; i++)
                for (int j = 0; j < jumlahKarakter - i - 1; j++)
                    if (dbKarakter[j].namaKarakter < dbKarakter[j +
1].namaKarakter)
                        swap(dbKarakter[j], dbKarakter[j + 1]);
            cout << "Berhasil diurutkan!\n";
            cout << "\nTekan Enter untuk melihat hasil"; cin.ignore();
cin.get();
            tampilkanSemuaKarakter(dbKarakter, jumlahKarakter, roleSaatIni);
        } else if (pil == 2) {
            for (int i = 0; i < jumlahKarakter - 1; i++) {
                int minIdx = i;
                for (int j = i + 1; j < jumlahKarakter; j++)
                    if (dbKarakter[j].idKarakter <
dbKarakter[minIdx].idKarakter)
                        minIdx = j;
            }
        }
    }
}

```

```

        swap(dbKarakter[i], dbKarakter[minIdx]);
    }
    cout << "Berhasil diurutkan!\n";
    cout << "\nTekan Enter untuk melihat hasil"; cin.ignore();
cin.get();
    tampilkanSemuaKarakter(dbKarakter, jumlahKarakter, roleSaatIni);
} else if (pil == 3) {
    for (int i = 1; i < jumlahKarakter; i++) {
        Karakter key = dbKarakter[i];
        int j = i - 1;
        while (j >= 0 && dbKarakter[j].detail.elemenKarakter >
key.detail.elemenKarakter) {
            dbKarakter[j + 1] = dbKarakter[j];
            j--;
        }
        dbKarakter[j + 1] = key;
    }
    cout << "Berhasil diurutkan!\n";
    cout << "\nTekan Enter untuk melihat hasil"; cin.ignore();
cin.get();
    tampilkanSemuaKarakter(dbKarakter, jumlahKarakter, roleSaatIni);
}
// pil == 4 => kembali, tidak ada aksi
} catch (const invalid_argument& e) {
    cout << "\n[ERROR] " << e.what() << "\n";
} catch (const out_of_range& e) {
    cout << "\n[ERROR] " << e.what() << "\n";
}
}

void tambahKarakter(Karakter dbKarakter[], int &jumlahKarakter) {
    clearScreen();
    try {
        if (jumlahKarakter >= 100)
            throw length_error("Database penuh! Kapasitas maksimal 100
karakter.");

        Karakter &charBaru = dbKarakter[jumlahKarakter];
        charBaru.idKarakter = dbKarakter[jumlahKarakter - 1].idKarakter + 1;

        cout << "\n--- TAMBAH KARAKTER ---\n";
        cout << "Nama Karakter    : "; cin.ignore(); getline(cin,
charBaru.namaKarakter);
        validasiStringKosong(charBaru.namaKarakter, "Nama");

        cout << "Rank (S/A)      : "; getline(cin, charBaru.rankKarakter);
        validasiRank(charBaru.rankKarakter);

        cout << "Role          : "; getline(cin,
charBaru.detail.roleKarakter);
        validasiStringKosong(charBaru.detail.roleKarakter, "Role");
    }
}

```

```

        cout << "Elemen          : "; getline(cin,
charBaru.detail.elemenKarakter);
        validasiStringKosong(charBaru.detail.elemenKarakter, "Elemen");

        cout << "Tier (T0/T0.5/T1/T2/T3): "; getline(cin,
charBaru.tierKarakter);
        validasiTier(charBaru.tierKarakter);

        jumlahKarakter++;
        cout << "Karakter berhasil ditambahkan!\n";

    } catch (const length_error& e) {
        cout << "\n[ERROR] " << e.what() << "\n";
    } catch (const invalid_argument& e) {
        cout << "\n[ERROR] " << e.what() << "\n";
        cout << "Penambahan karakter dibatalkan.\n";
    }
}

void editKarakter(Karakter dbKarakter[], int jumlahKarakter) {
    clearScreen();
    try {
        int idEdit = inputInt("\n--- EDIT KARAKTER ---\nMasukkan ID Karakter
yang ingin diedit: ");
        validasiID(idEdit);

        int i = interpolationSearchID(dbKarakter, jumlahKarakter, &idEdit);
        if (i == -1) throw runtime_error("ID Karakter tidak ditemukan.");

        Karakter &kEdit = dbKarakter[i];
        bool sedangEdit = true;
        do {
            clearScreen();
            cout << "\n--- EDIT KARAKTER (" << kEdit.namaKarakter << ") ---\n";
            cout << "1. Nama      : " << kEdit.namaKarakter << "\n";
            cout << "2. Rank      : " << kEdit.rankKarakter << "\n";
            cout << "3. Role      : " << kEdit.detail.roleKarakter << "\n";
            cout << "4. Elemen    : " << kEdit.detail.elemenKarakter << "\n";
            cout << "5. Tier      : " << kEdit.tierKarakter << "\n";
            cout << "6. Kembali\n";

            try {
                int pilihanEdit = inputInt("Pilih data yang ingin diubah (1-6):
");

                validasiMenu(pilihanEdit, 1, 6);

                if (pilihanEdit == 1) {
                    cout << "Masukkan Nama Baru: "; cin.ignore(); getline(cin,
kEdit.namaKarakter);
                    validasiStringKosong(kEdit.namaKarakter, "Nama");

```

```

        cout << "Nama berhasil diubah.\n";
    } else if (pilihanEdit == 2) {
        cout << "Masukkan Rank Baru (S/A): "; cin.ignore();
getline(cin, kEdit.rankKarakter);
        validasiRank(kEdit.rankKarakter);
    } else if (pilihanEdit == 3) {
        cout << "Masukkan Role Baru: "; cin.ignore(); getline(cin,
kEdit.detail.roleKarakter);
        validasiStringKosong(kEdit.detail.roleKarakter, "Role");
    } else if (pilihanEdit == 4) {
        cout << "Masukkan Elemen Baru: "; cin.ignore(); getline(cin,
kEdit.detail.elemenKarakter);
        validasiStringKosong(kEdit.detail.elemenKarakter, "Elemen");
    } else if (pilihanEdit == 5) {
        cout << "Masukkan Tier Baru: "; cin.ignore(); getline(cin,
kEdit.tierKarakter);
        validasiTier(kEdit.tierKarakter);
    } else if (pilihanEdit == 6) {
        sedangEdit = false;
    }
} catch (const exception& e) {
    cout << "\n[ERROR] " << e.what() << "\n";
    cout << "Tekan Enter untuk mencoba lagi"; cin.ignore();
cin.get();
}
} while (sedangEdit);

} catch (const invalid_argument& e) {
    cout << "\n[ERROR] " << e.what() << "\n";
} catch (const runtime_error& e) {
    cout << "\n[ERROR] " << e.what() << "\n";
}
}

void hapusKarakter(Karakter dbKarakter[], int &jumlahKarakter) {
    clearScreen();
    try {
        int idHapus = inputInt("\n--- HAPUS KARAKTER ---\nMasukkan ID Karakter
yang ingin dihapus: ");
        validasiID(idHapus);

        int pos = interpolationSearchID(dbKarakter, jumlahKarakter, &idHapus);
        if (pos == -1) throw runtime_error("ID Karakter tidak ditemukan.");

        for (int j = pos; j < jumlahKarakter - 1; j++)
            dbKarakter[j] = dbKarakter[j + 1];
        jumlahKarakter--;
        cout << "Karakter berhasil dihapus!\n";

    } catch (const invalid_argument& e) {
        cout << "\n[ERROR] " << e.what() << "\n";
    }
}

```

```

    } catch (const runtime_error& e) {
        cout << "\n[ERROR] " << e.what() << "\n";
    }
}

void tampilkanTierlist(Karakter dbKarakter[], int jumlahKarakter) {
    clearScreen();
    string daftarRole[6] = {"Attack", "Stun", "Support", "Anomaly", "Defense",
    "Rupture"};
    string daftarTier[5] = {"T0", "T0.5", "T1", "T2", "T3"};
    for (int r = 0; r < 6; r++) {
        cout << "\n=== CLASS : " << daftarRole[r] << " ===\n";
        for (int t = 0; t < 5; t++) {
            bool ada = false;
            for (int i = 0; i < jumlahKarakter; i++) {
                if (dbKarakter[i].detail.roleKarakter == daftarRole[r] &&
                dbKarakter[i].tierKarakter == daftarTier[t]) {
                    if (!ada) { cout << " [ " << daftarTier[t] << " ]\n"; ada =
                    true; }
                    cout << " - " << dbKarakter[i].namaKarakter << " (Elemen: "
                    << dbKarakter[i].detail.elemenKarakter << ")\n";
                }
            }
        }
    }
}

void ubahTierKarakter(Karakter dbKarakter[], int jumlahKarakter) {
    clearScreen();
    try {
        int idTierEdit = inputInt("\n--- UBAH TIER KARAKTER ---\nMasukkan ID
        Karakter: ");
        validasiID(idTierEdit);

        int i = interpolationSearchID(dbKarakter, jumlahKarakter, &idTierEdit);
        if (i == -1) throw runtime_error("ID Karakter tidak valid.");

        Karakter &kEdit = dbKarakter[i];
        cout << "Karakter: " << kEdit.namaKarakter << " (Tier: " <<
        kEdit.tierKarakter << ")\n";

        int pilihanTier = inputInt("Pilih Tier Baru:\n1. T0\n2. T0.5\n3. T1\n4.
        T2\n5. T3\nPilihan: ");
        validasiMenu(pilihanTier, 1, 5);

        string tierBaru[] = {"T0", "T0.5", "T1", "T2", "T3"};
        kEdit.tierKarakter = tierBaru[pilihanTier - 1];
        cout << "Tier berhasil diperbarui menjadi " << kEdit.tierKarakter <<
        "\n";

    } catch (const invalid_argument& e) {

```



```

        cout << "\n[ERROR] " << e.what() << "\n";
    } catch (const out of range& e) {
        cout << "\n[ERROR] " << e.what() << "\n";
    } catch (const runtime_error& e) {
        cout << "\n[ERROR] " << e.what() << "\n";
    }
}

void editProfil(AkunUser &userAktif) {
    clearScreen();
    try {
        cout << "\n= EDIT PROFIL =\n";
        cout << "Username saat ini: " << userAktif.namaUser << "\n";

        string namaBaru, passBaru;
        cout << "Masukkan Username baru: "; cin >> namaBaru;
        validasiStringKosong(namaBaru, "Username");

        cout << "Masukkan Password baru: "; cin >> passBaru;
        validasiStringKosong(passBaru, "Password");

        userAktif.namaUser = namaBaru;
        userAktif.password = passBaru;
        cout << "Profil berhasil diperbarui!\n";

    } catch (const invalid_argument& e) {
        cout << "\n[ERROR] " << e.what() << "\n";
        cout << "Perubahan profil dibatalkan.\n";
    }
}

void manajemenUser(AkunUser dbUser[], int &jumlahUser, int idUserAktif) {
    bool menuUserAktif = true;
    do {
        clearScreen();
        cout << "\n= MANAJEMEN USER =\n";
        cout << "1. Lihat Semua User\n";
        cout << "2. Ubah Data User\n";
        cout << "3. Hapus User\n";
        cout << "4. Kembali\n";

        try {
            int menuUser = inputInt("Pilih aksi: ");
            validasiMenu(menuUser, 1, 4);

            if (menuUser == 1) {
                clearScreen();
                cout << "\n" << left << setw(5) << "ID" << setw(20) <<
"Username" << setw(15) << "Role" << "\n";
                cout << "-----\n";
                for (int i = 0; i < jumlahUser; i++)

```

```

        cout << left << setw(5) << dbUser[i].idUser << setw(20) <<
dbUser[i].namaUser
        << setw(15) << dbUser[i].roleUser << "\n";
        cout << "\nTekan Enter untuk melanjutkan"; cin.ignore();
cin.get();

    } else if (menuUser == 2) {
        try {
            int idEdit = inputInt("Masukkan ID User yang ingin diubah:
");

            validasiID(idEdit);
            bool ketemu = false;
            for (int i = 0; i < jumlahUser; i++) {
                if (dbUser[i].idUser == idEdit) {
                    ketemu = true;
                    string namaBaru, passBaru;
                    cout << "Username Baru: "; cin >> namaBaru;
                    validasiStringKosong(namaBaru, "Username");
                    cout << "NIM/Password Baru: "; cin >> passBaru;
                    validasiStringKosong(passBaru, "Password");
                    dbUser[i].namaUser = namaBaru;
                    dbUser[i].password = passBaru;
                    cout << "Data berhasil diperbarui!\n";
                    break;
                }
            }
            if (!ketemu) throw runtime_error("ID User tidak
ditemukan.");
        } catch (const exception& e) {
            cout << "\n[ERROR] " << e.what() << "\n";
        }
        cout << "\nTekan Enter untuk melanjutkan"; cin.ignore();
cin.get();

    } else if (menuUser == 3) {
        try {
            int idHapus = inputInt("Masukkan ID User yang ingin dihapus:
");

            validasiID(idHapus);
            if (idHapus == idUserAktif)
                throw runtime_error("Tidak bisa menghapus akun yang
sedang aktif!");

            bool ketemu = false;
            for (int i = 0; i < jumlahUser; i++) {
                if (dbUser[i].idUser == idHapus) {
                    ketemu = true;
                    for (int j = i; j < jumlahUser - 1; j++)
                        dbUser[j] = dbUser[j + 1];
                    jumlahUser--;
                    cout << "User berhasil dihapus!\n";
                    break;
                }
            }
        }
    }
}

```

```

        }
    }
    if (!ketemu) throw runtime_error("ID User tidak
ditemukan.");
    } catch (const exception& e) {
        cout << "\n[ERROR] " << e.what() << "\n";
    }
    cout << "\nTekan Enter untuk melanjutkan"; cin.ignore();
cin.get();

    } else if (menuUser == 4) {
        menuUserAktif = false;
    }

    } catch (const invalid_argument& e) {
        cout << "\n[ERROR] " << e.what() << "\n";
        cin.ignore(); cin.get();
    } catch (const out_of_range& e) {
        cout << "\n[ERROR] " << e.what() << "\n";
        cin.ignore(); cin.get();
    }
} while (menuUserAktif);
}

void subMenuDbKarakter(Karakter dbKarakter[], int &jumlahKarakter, string
roleSaatIni) {
    bool sesiDbKarakter = true;
    do {
        clearScreen();
        cout << "\n= DATABASE KARAKTER =\n";
        cout << "1. Tampilkan Semua Karakter\n";
        cout << "2. Urutkan Karakter (Sorting)\n";
        cout << "3. Cari Karakter\n";
        if (roleSaatIni == "admin") {
            cout << "4. Tambah Karakter Baru\n";
            cout << "5. Edit Karakter\n";
            cout << "6. Hapus Karakter\n";
            cout << "7. Kembali\n";
        } else {
            cout << "4. Kembali\n";
        }
    }

    try {
        int maxMenu = (roleSaatIni == "admin") ? 7 : 4;
        int menuDb = inputInt("Pilih aksi: ");
        validasiMenu(menuDb, 1, maxMenu);

        if (menuDb == 1) {
            tampilkanSemuaKarakter(dbKarakter, jumlahKarakter, roleSaatIni);
            cout << "\nTekan Enter untuk kembali"; cin.ignore(); cin.get();
        } else if (menuDb == 2) {

```

```

        menuSortingKarakter(dbKarakter, jumlahKarakter, roleSaatIni);
        cout << "\nTekan Enter untuk kembali";
        if (cin.peek() == '\n') cin.ignore();
        cin.get();
    } else if (menuDb == 3) {
        clearScreen();
        cout << "\n= CARI KARAKTER =\n";
        cout << "1. Cari berdasarkan ID (Interpolation Search)\n";
        cout << "2. Cari berdasarkan Nama (Jump Search)\n";
        try {
            int pilCari = inputInt("Pilih metode pencarian: ");
            validasiMenu(pilCari, 1, 2);
            int hasilPencarian = -1;

            if (pilCari == 1) {
                int idCari = inputInt("Masukkan ID Karakter: ");
                validasiID(idCari);
                hasilPencarian = interpolationSearchID(dbKarakter,
jumlahKarakter, &idCari);
            } else {
                string keyword;
                cout << "Masukkan Nama Karakter (Perhatikan Kapital): ";
                cin.ignore(); getline(cin, keyword);
                validasiStringKosong(keyword, "Nama Karakter");
                hasilPencarian = jumpSearchNama(dbKarakter,
jumlahKarakter, &keyword);
            }

            cout << "\n= HASIL PENCARIAN =\n";
            if (hasilPencarian != -1) {
                Karakter &kFound = dbKarakter[hasilPencarian];
                if (roleSaatIni == "admin") cout << "ID      : " <<
kFound.idKarakter << "\n";
                cout << "Nama    : " << kFound.namaKarakter << "\n";
                cout << "Rank    : " << kFound.rankKarakter << "\n";
                cout << "Role    : " << kFound.detail.roleKarakter <<
"\n";

                cout << "Elemen : " << kFound.detail.elemenKarakter <<
"\n";

                if (roleSaatIni == "admin") cout << "Tier     : " <<
kFound.tierKarakter << "\n";
            } else {
                cout << "Karakter tidak ditemukan.\n";
            }
        } catch (const exception& e) {
            cout << "\n[ERROR] " << e.what() << "\n";
        }
        cout << "\nTekan Enter untuk kembali"; cin.ignore(); cin.get();

    } else if (menuDb == 4 && roleSaatIni == "admin") {
        tambahKarakter(dbKarakter, jumlahKarakter);
    }
}

```

```

        cout << "\nTekan Enter untuk kembali"; cin.get();
    } else if (menuDb == 5 && roleSaatIni == "admin") {
        editKarakter(dbKarakter, jumlahKarakter);
    } else if (menuDb == 6 && roleSaatIni == "admin") {
        hapusKarakter(dbKarakter, jumlahKarakter);
        cout << "\nTekan Enter untuk kembali"; cin.ignore(); cin.get();
    } else if ((menuDb == 7 && roleSaatIni == "admin") || (menuDb == 4
&& roleSaatIni == "user")) {
        sesiDbKarakter = false;
    }

    } catch (const invalid_argument& e) {
        cout << "\n[ERROR] " << e.what() << "\n";
        cin.ignore(); cin.get();
    } catch (const out_of_range& e) {
        cout << "\n[ERROR] " << e.what() << "\n";
        cin.ignore(); cin.get();
    }
} while (sesiDbKarakter);
}

void subMenuTierlist(Karakter dbKarakter[], int &jumlahKarakter, string
roleSaatIni) {
    bool sesiTierlist = true;
    do {
        clearScreen();
        cout << "\n= TIERLIST KARAKTER =\n";
        cout << "1. Tampilkan Tierlist Berdasarkan Class/Role\n";
        if (roleSaatIni == "admin") {
            cout << "2. Ubah Tier Karakter\n";
            cout << "3. Kembali\n";
        } else {
            cout << "2. Kembali\n";
        }
    }

    try {
        int maxMenu = (roleSaatIni == "admin") ? 3 : 2;
        int menuTier = inputInt("Pilih aksi: ");
        validasiMenu(menuTier, 1, maxMenu);

        if (menuTier == 1) {
            tampilkanTierlist(dbKarakter, jumlahKarakter);
            cout << "\nTekan Enter untuk kembali"; cin.ignore(); cin.get();
        } else if (menuTier == 2 && roleSaatIni == "admin") {
            ubahTierKarakter(dbKarakter, jumlahKarakter);
            cout << "\nTekan Enter untuk kembali"; cin.ignore(); cin.get();
        } else if ((menuTier == 3 && roleSaatIni == "admin") || (menuTier ==
2 && roleSaatIni == "user")) {
            sesiTierlist = false;
        }
    }
}

```

```

        } catch (const invalid argument& e) {
            cout << "\n[ERROR] " << e.what() << "\n";
            cin.ignore(); cin.get();
        } catch (const out of range& e) {
            cout << "\n[ERROR] " << e.what() << "\n";
            cin.ignore(); cin.get();
        }
    } while (sesiTierlist);
}

void Admin(Karakter dbKarakter[], int &jumlahKarakter, AkunUser dbUser[], int
&jumlahUser, int indeksUserAktif, bool &dashboardAktif) {
    do {
        clearScreen();
        cout << "\nSelamat datang, " << dbUser[indeksUserAktif].namaUser << "
(Admin)!\n";
        cout << "\n=== DASHBOARD ADMIN ===\n";
        cout << "1. Database Karakter\n";
        cout << "2. Tierlist Karakter\n";
        cout << "3. Manajemen User\n";
        cout << "4. Logout\n";

        try {
            int menuDashboard = inputInt("Pilih menu: ");
            validasiMenu(menuDashboard, 1, 4);

            if (menuDashboard == 1) subMenuDbKarakter(dbKarakter,
jumlahKarakter, "admin");
            else if (menuDashboard == 2) subMenuTierlist(dbKarakter,
jumlahKarakter, "admin");
            else if (menuDashboard == 3) manajemenUser(dbUser, jumlahUser,
dbUser[indeksUserAktif].idUser);
            else if (menuDashboard == 4) { cout << "Logout berhasil!\n";
dashboardAktif = false; }

        } catch (const invalid argument& e) {
            cout << "\n[ERROR] " << e.what() << "\n";
            cin.ignore(); cin.get();
        } catch (const out of range& e) {
            cout << "\n[ERROR] " << e.what() << "\n";
            cin.ignore(); cin.get();
        }
    } while (dashboardAktif);
}

void User(Karakter dbKarakter[], int &jumlahKarakter, AkunUser dbUser[], int
indeksUserAktif, bool &dashboardAktif) {
    do {
        clearScreen();
        cout << "\nSelamat datang, " << dbUser[indeksUserAktif].namaUser <<
"! \n";
    }

```

```

        cout << "\n=== DASHBOARD PENGGUNA ===\n";
        cout << "1. Database Karakter\n";
        cout << "2. Tierlist Karakter\n";
        cout << "3. Edit Profil\n";
        cout << "4. Logout\n";

        try {
            int menuDashboard = inputInt("Pilih menu: ");
            validasiMenu(menuDashboard, 1, 4);

            if (menuDashboard == 1) subMenuDbKarakter(dbKarakter,
jumlahKarakter, "user");
            else if (menuDashboard == 2) subMenuTierlist(dbKarakter,
jumlahKarakter, "user");
            else if (menuDashboard == 3) {
                editProfil(dbUser[indeksUserAktif]);
                cout << "\nTekan Enter untuk kembali"; cin.ignore(); cin.get();
            } else if (menuDashboard == 4) { cout << "Logout berhasil!\n";
dashboardAktif = false; }

        } catch (const invalid_argument& e) {
            cout << "\n[ERROR] " << e.what() << "\n";
            cin.ignore(); cin.get();
        } catch (const out_of_range& e) {
            cout << "\n[ERROR] " << e.what() << "\n";
            cin.ignore(); cin.get();
        }
    } while (dashboardAktif);
}

void registerUser(AkunUser dbUser[], int &jumlahUser) {
    clearScreen();
    try {
        if (jumlahUser >= 50) throw length_error("Kapasitas user penuh! Maksimal
50 akun.");

        cout << "\n--- MENU REGISTER ---\n";
        AkunUser &akunBaru = dbUser[jumlahUser];
        akunBaru.idUser = (jumlahUser == 0) ? 1 : dbUser[jumlahUser - 1].idUser
+ 1;

        cout << "Masukkan Username: "; cin >> akunBaru.namaUser;
        validasiStringKosong(akunBaru.namaUser, "Username");
        cout << "Masukkan Password: "; cin >> akunBaru.password;
        validasiStringKosong(akunBaru.password, "Password");

        akunBaru.roleUser = "user";
        jumlahUser++;
        cout << "Register berhasil! Silakan Login.\n";

    } catch (const length_error& e) {

```

```

        cout << "\n[ERROR] " << e.what() << "\n";
    } catch (const invalid_argument& e) {
        cout << "\n[ERROR] " << e.what() << "\n";
        cout << "Register dibatalkan.\n";
    }
}

void prosesLogin(Karakter dbKarakter[], int &jumlahKarakter, AkunUser dbUser[],
int &jumlahUser, bool &App) {
    string inputUser, inputPassword;
    bool statusLoginSukses = false;
    int indeksUserAktif = -1;
    int cobaLogin = 3;

    while (cobaLogin > 0 && !statusLoginSukses) {
        clearScreen();
        cout << "\nLogin (Sisa percobaan: " << cobaLogin << ")\n";
        cout << "Masukkan Username: "; cin >> inputUser;
        cout << "Masukkan password : "; cin >> inputPassword;

        for (int i = 0; i < jumlahUser; i++) {
            if (dbUser[i].namaUser == inputUser && dbUser[i].password ==
inputPassword) {
                statusLoginSukses = true;
                indeksUserAktif = i;
                break;
            }
        }

        if (!statusLoginSukses) {
            cout << "Username atau Password salah!\n";
            cobaLogin--;
            cout << "\nTekan Enter untuk mencoba lagi"; cin.ignore(); cin.get();
        }
    }

    if (cobaLogin == 0) {
        clearScreen();
        cout << "\n===== \n";
        cout << "  Program dihentikan. Gagal login 3 kali.  \n";
        cout << "===== \n";
        App = false;
        return;
    }

    if (statusLoginSukses) {
        bool dashboardAktif = true;
        string roleSaatIni = dbUser[indeksUserAktif].roleUser;

        if (roleSaatIni == "admin")
            Admin(dbKarakter, jumlahKarakter, dbUser, jumlahUser,

```



```

indeksUserAktif, dashboardAktif);
    else if (roleSaatIni == "user")
        User(dbKarakter, jumlahKarakter, dbUser, indeksUserAktif,
dashboardAktif);
    else {
        cout << "\nHeker Jir\n";
        App = false;
    }
}
}

int main() {
    Karakter dbKarakter[100];
    int jumlahKarakter = 0;
    AkunUser dbUser[50];
    int jumlahUser = 0;

    inisialisasiData(dbKarakter, jumlahKarakter, dbUser, jumlahUser);

    bool App = true;
    do {
        clearScreen();
        cout << "=====\n";
        cout << "    DATABASE & TIERLIST ZENLESS ZONE ZERO    \n";
        cout << "=====\n";
        cout << "\n--- MENU LOG IN ---\n";
        cout << "1. Login\n";
        cout << "2. Register\n";
        cout << "3. Keluar Aplikasi\n";

        try {
            int menuAwal = inputInt("Pilih menu (1-3): ");
            validasiMenu(menuAwal, 1, 3);

            if (menuAwal == 3) {
                cout << "Terima kasih telah menggunakan sistem database ZZZ\n";
                App = false;
            } else if (menuAwal == 2) {
                registerUser(dbUser, jumlahUser);
                cout << "\nTekan Enter untuk kembali"; cin.ignore(); cin.get();
            } else if (menuAwal == 1) {
                prosesLogin(dbKarakter, jumlahKarakter, dbUser, jumlahUser,
App);
            }

        } catch (const invalid_argument& e) {
            cout << "\n[ERROR] " << e.what() << "\n";
            cout << "\nTekan Enter untuk kembali"; cin.ignore(); cin.get();
        } catch (const out_of_range& e) {
            cout << "\n[ERROR] " << e.what() << "\n";
            cout << "\nTekan Enter untuk kembali"; cin.ignore(); cin.get();
        }
    } while (App);
}

```

```

    }
} while (App);

return 0;
}

```

B. .H

```

#ifndef VALIDASI_H
#define VALIDASI_H

#include <iostream>
#include <string>
#include <stdexcept>
#include <limits>

using namespace std;

int inputInt(const string& prompt) {
    int nilai;
    cout << prompt;
    if (!(cin >> nilai)) {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        throw invalid_argument("Input harus berupa angka bulat!");
    }
    return nilai;
}

void validasiMenu(int pilihan, int min, int max) {
    if (pilihan < min || pilihan > max) {
        throw out_of_range("Pilihan menu tidak valid! Masukkan angka " +
                           to_string(min) + " - " + to_string(max) + ".");
    }
}

void validasiStringKosong(const string& input, const string& namaField) {
    if (input.empty()) {
        throw invalid_argument("Field '" + namaField + "' tidak boleh kosong!");
    }
}

void validasiRank(const string& rank) {
    if (rank != "S" && rank != "A") {
        throw invalid_argument("Rank tidak valid! Harus 'S' atau 'A'.");
    }
}

```

```
void validasiTier(const string& tier) {
    if (tier != "T0" && tier != "T0.5" && tier != "T1" && tier != "T2" && tier
    != "T3") {
        throw invalid_argument("Tier tidak valid! Gunakan: T0, T0.5, T1, T2,
    atau T3.");
    }
}

void validasiID(int id) {
    if (id <= 0) {
        throw invalid_argument("ID harus berupa angka positif!");
    }
}

#endif
```

4. Hasil Output

1. Register

```
--- MENU REGISTER ---  
Masukkan Nama (Username): wow  
Masukkan NIM (Password): 123  
Register berhasil! Silakan Login.
```

2. Login

```
--- MENU LOG IN ---  
1. Login  
2. Register  
3. Keluar Aplikasi  
Pilih menu (1-3): 1  
  
Login (Sisa percobaan: 3)  
Masukkan Nama: ajis  
Masukkan NIM (Password): 048  
  
Selamat datang, ajis!
```

3. Dashboard(Admin)

```
=== DASHBOARD ADMIN ===  
1. Database Karakter  
2. Tierlist Karakter  
3. Manajemen User  
4. Logout  
Pilih menu: |
```

4. Dashboard(User)

```
=== DASHBOARD PENGGUNA ===  
1. Database Karakter  
2. Tierlist Karakter  
3. Edit Profil  
4. Logout  
Pilih menu: |
```

5. Database Character(Admin)

```
= DATABASE KARAKTER =  
1. Tampilkan Semua Karakter  
2. Cari Karakter  
3. Tambah Karakter Baru  
4. Edit Karakter  
5. Hapus Karakter  
6. Kembali  
Pilih aksi: |
```

6. Database Character(User)

```
= DATABASE KARAKTER =  
1. Tampilkan Semua Karakter  
2. Cari Karakter  
3. Kembali  
Pilih aksi: |
```

7. Database Character: Tampilkan semua Karakter(Admin & User)

ID	Nama	Rank	Role	Elemen	Tier
1	Alice	S	Anomaly	Physical	T0.5
2	Anby Demara	A	Stun	Electric	T3
3	Anby: Soldier 0	S	Attack	Electric	T0.5
4	Anton	A	Attack	Electric	T2
5	Aria	S	Anomaly	Ether	T0.5
6	Astra Yao	S	Support	Ether	T0
7	Banyue	S	Rupture	Fire	T0.5
8	Ben	S	Defense	Fire	T3
9	Billy	A	Attack	Physical	T2
10	Burnice	S	Anomaly	Fire	T1
11	Caesar	S	Defense	Physical	T2
12	Corin	A	Attack	Physical	T2
13	Dialyn	S	Stun	Physical	T0
14	Ellen	S	Attack	Ice	T1
15	Evelyn	S	Attack	Fire	T0.5
16	Grace	S	Anomaly	Electric	T2
17	Harumasa	S	Attack	Electric	T1
18	Hugo	S	Attack	Ice	T1
19	Jane Doe	S	Anomaly	Physical	T1
20	Ju Fufu	S	Stun	Fire	T0.5
21	Koleda	S	Stun	Fire	T2
22	Lighter	S	Stun	Fire	T0.5
23	Lucia	S	Support	Ether	T0
24	Lucy	A	Support	Fire	T2
25	Lycaon	S	Stun	Ice	T1
26	Manato	A	Rupture	Fire	T1
27	Miyabi	S	Anomaly	Frost	T0
28	Nekomata	S	Attack	Physical	T2
29	Nicole	A	Support	Ether	T0.5
30	Orphie & Magus	S	Attack	Fire	T0.5
31	Pan Yinhu	A	Defense	Physical	T1
32	Piper	A	Anomaly	Physical	T1
33	Pulchra	A	Stun	Physical	T2
34	Qingyi	S	Stun	Electric	T1
35	Rina	S	Support	Electric	T2
36	Seed	S	Attack	Electric	T0.5
37	Seth	A	Defense	Electric	T3
38	Soldier 11	S	Attack	Fire	T1
39	Soukaku	A	Support	Ice	T1
40	Sunna	S	Support	Physical	T0
41	Trigger	S	Stun	Electric	T0.5
42	Vivian	S	Anomaly	Ether	T0.5
43	Yanagi	S	Anomaly	Electric	T1
44	Ye Shunguang	S	Attack	Honed Edge	T0
45	Yidhari	S	Rupture	Ice	T0.5
46	Yixuan	S	Rupture	Auric Ink	T0
47	Yuzuha	S	Support	Physical	T0
48	Zhao	S	Defense	Ice	T0.5
49	Zhu Yuan	S	Attack	Ether	T1

8. Database Character: Cari Karakter(Admin)

```
= DATABASE KARAKTER =
1. Tampilkan Semua Karakter
2. Cari Karakter
3. Tambah Karakter Baru
4. Edit Karakter
5. Hapus Karakter
6. Kembali
Pilih aksi: 2
Masukkan Nama Karakter yang dicari (Perhatikan Huruf Besar/Kecil): Ellen

= HASIL PENCARIAN =
ID      : 14
Nama    : Ellen
Rank    : S
Role    : Attack
Elemen  : Ice
Tier    : T1
```

9. Database Character: Cari Karakter(User)

```
= DATABASE KARAKTER =
1. Tampilkan Semua Karakter
2. Cari Karakter
3. Kembali
Pilih aksi: 2
Masukkan Nama Karakter yang dicari (Perhatikan Huruf Besar/Kecil): Ellen

= HASIL PENCARIAN =
Nama    : Ellen
Rank    : S
Role    : Attack
Elemen  : Ice
```

10. Database Character: Create Karakter(Admin)

```
= DATABASE KARAKTER =
1. Tampilkan Semua Karakter
2. Cari Karakter
3. Tambah Karakter Baru
4. Edit Karakter
5. Hapus Karakter
6. Kembali
Pilih aksi: 3
Nama Karakter: Cissia
Rank Karakter (S/A): S
Role (Attack/Stun/Support/Anomaly/Defense/Rupture): Attack
Elemen (Ice/Ether/Electric/dll): Electric
Tier (T0/T0.5/T1/T2/T3): T1
Karakter berhasil ditambahkan!
```

```
Masukkan Nama Karakter yang dicari (Perhatikan Huruf Besar/Kecil): Cissia

= HASIL PENCARIAN =
ID      : 50
Nama    : Cissia
Rank    : S
Role    : Attack
Elemen  : Electric
Tier    : T1
```

11. Database Character: Edit Karakter(Admin)

```
= DATABASE KARAKTER =
1. Tampilkan Semua Karakter
2. Cari Karakter
3. Tambah Karakter Baru
4. Edit Karakter
5. Hapus Karakter
6. Kembali
Pilih aksi: 4
Masukkan ID Karakter yang ingin diedit: 50

--- EDIT KARAKTER (Cissia) ---
1. Nama      : Cissia
2. Rank      : S
3. Role      : Attack
4. Elemen    : Electric
5. Tier      : T1
6. Kembali
Pilih data yang ingin diubah (1-6): 2
Masukkan Rank Baru (S/A): A
Rank berhasil diubah

--- EDIT KARAKTER (Cissia) ---
1. Nama      : Cissia
2. Rank      : A
3. Role      : Attack
4. Elemen    : Electric
5. Tier      : T1
6. Kembali
Pilih data yang ingin diubah (1-6): |
```


12. Database Character: Hapus Karakter(Admin)

```
= DATABASE KARAKTER =  
1. Tampilkan Semua Karakter  
2. Cari Karakter  
3. Tambah Karakter Baru  
4. Edit Karakter  
5. Hapus Karakter  
6. Kembali  
Pilih aksi: 5  
Masukkan ID Karakter yang ingin dihapus: 50  
Karakter berhasil dihapus!  
  
= DATABASE KARAKTER =  
1. Tampilkan Semua Karakter  
2. Cari Karakter  
3. Tambah Karakter Baru  
4. Edit Karakter  
5. Hapus Karakter  
6. Kembali  
Pilih aksi: 2  
Masukkan Nama Karakter yang dicari (Perhatikan Huruf Besar/Kecil): Cissia  
  
= HASIL PENCARIAN =  
Karakter tidak ditemukan.
```

13. Tierlist(Admin)

```
= TIERLIST KARAKTER =  
1. Tampilkan Tierlist Berdasarkan Class/Role  
2. Ubah Tier Karakter  
3. Kembali  
Pilih aksi: |
```

14. Tierlist(User)

```
= TIERLIST KARAKTER =  
1. Tampilkan Tierlist Berdasarkan Class/Role  
2. Kembali  
Pilih aksi: |
```

14. Tierlist: Tampilkan Tierlist(Admin & User)

```
=== CLASS : Attack ===
[ T0 ]
- Ye Shunguang (Elemen: Honed Edge)
[ T0.5 ]
- Anby: Soldier 0 (Elemen: Electric)
- Evelyn (Elemen: Fire)
- Orphie & Magus (Elemen: Fire)
- Seed (Elemen: Electric)
[ T1 ]
- Ellen (Elemen: Ice)
- Harumasa (Elemen: Electric)
- Hugo (Elemen: Ice)
- Soldier 11 (Elemen: Fire)
- Zhu Yuan (Elemen: Ether)
[ T2 ]
- Anton (Elemen: Electric)
- Billy (Elemen: Physical)
- Corin (Elemen: Physical)
- Nekomata (Elemen: Physical)

=== CLASS : Stun ===
[ T0 ]
- Dialyn (Elemen: Physical)
[ T0.5 ]
- Ju Fufu (Elemen: Fire)
- Lighter (Elemen: Fire)
- Trigger (Elemen: Electric)
[ T1 ]
- Lycaon (Elemen: Ice)
- Qingyi (Elemen: Electric)
[ T2 ]
- Koleda (Elemen: Fire)
- Pulchra (Elemen: Physical)
[ T3 ]
- Anby Demara (Elemen: Electric)

=== CLASS : Support ===
[ T0 ]
- Astra Yao (Elemen: Ether)
- Lucia (Elemen: Ether)
- Sunna (Elemen: Physical)
- Yuzuha (Elemen: Physical)
[ T0.5 ]
- Nicole (Elemen: Ether)
[ T1 ]
- Soukaku (Elemen: Ice)
[ T2 ]
- Lucy (Elemen: Fire)
- Rina (Elemen: Electric)
```

```

=== CLASS : Anomaly ===
[ T0 ]
- Miyabi (Elemen: Frost)
[ T0.5 ]
- Alice (Elemen: Physical)
- Aria (Elemen: Ether)
- Vivian (Elemen: Ether)
[ T1 ]
- Burnice (Elemen: Fire)
- Jane Doe (Elemen: Physical)
- Piper (Elemen: Physical)
- Yanagi (Elemen: Electric)
[ T2 ]
- Grace (Elemen: Electric)

=== CLASS : Defense ===
[ T0.5 ]
- Zhao (Elemen: Ice)
[ T1 ]
- Pan Yinhu (Elemen: Physical)
[ T2 ]
- Caesar (Elemen: Physical)
[ T3 ]
- Ben (Elemen: Fire)
- Seth (Elemen: Electric)

=== CLASS : Rupture ===
[ T0 ]
- Yixuan (Elemen: Auric Ink)
[ T0.5 ]
- Banyue (Elemen: Fire)
- Yidhari (Elemen: Ice)
[ T1 ]
- Manato (Elemen: Fire)

```

15. Tierlist: Edit Tierlist(Admin)

```

= TIERLIST KARAKTER =
1. Tampilkan Tierlist Berdasarkan Class/Role
2. Ubah Tier Karakter
3. Kembali
Pilih aksi: 2
Masukkan ID Karakter : 49
Karakter saat ini : Zhu Yuan (Tier: T1)
Pilih Tier Baru:
1. T0
2. T0.5
3. T1
4. T2
5. T3
Pilihan: 4
Tier berhasil diperbarui menjadi T2

```

16. Manajemen User(Admin)

```
= MANAJEMEN USER =  
1. Lihat Semua User  
2. Ubah Data User  
3. Hapus User  
4. Kembali  
Pilih aksi: |
```

17. Manajemen User: Tampilkan semua User(Admin)

ID	Username	Role
1	ajis	admin
2	user	user
3	wow	user

18. Manajemen User: Ubah data User(Admin)

```
= MANAJEMEN USER =  
1. Lihat Semua User  
2. Ubah Data User  
3. Hapus User  
4. Kembali  
Pilih aksi: 2  
Masukkan ID User yang ingin diubah: 2  
Username Baru: sigma  
NIM/Password Baru: 048  
Data User berhasil diperbarui!
```

```
= MANAJEMEN USER =  
1. Lihat Semua User  
2. Ubah Data User  
3. Hapus User  
4. Kembali  
Pilih aksi: 1
```

ID	Username	Role
1	ajis	admin
2	sigma	user
3	wow	user

18. Manajemen User: Hapus User(Admin)

```
= MANAJEMEN USER =  
1. Lihat Semua User  
2. Ubah Data User  
3. Hapus User  
4. Kembali  
Pilih aksi: 3  
Masukkan ID User yang ingin dihapus: 2  
User berhasil dihapus!
```

```
= MANAJEMEN USER =  
1. Lihat Semua User  
2. Ubah Data User  
3. Hapus User  
4. Kembali  
Pilih aksi: 1
```

ID	Username	Role
1	ajis	admin
3	wow	user

18. Edit Profil (User)

```
=== DASHBOARD PENGGUNA ===  
1. Database Karakter  
2. Tierlist Karakter  
3. Edit Profil  
4. Logout  
Pilih menu: 3  
  
= EDIT PROFIL =  
Nama Username saat ini: wow  
Masukkan Nama Username baru: sigma  
Masukkan NIM/Password baru: 048  
Profil berhasil diperbarui!
```

ID	Username	Role
1	ajis	admin
3	sigma	user

19. Cari Karakter (User & Admin)

```
= CARI KARAKTER =  
1. Cari berdasarkan ID (Interpolation Search)  
2. Cari berdasarkan Nama (Jump Search)  
Pilih metode pencarian: 1  
Masukkan ID Karakter: 32  
  
= HASIL PENCARIAN =  
ID      : 32  
Nama    : Piper  
Rank    : A  
Role    : Anomaly  
Elemen  : Physical  
Tier    : T1  
  
Tekan Enter untuk kembali
```

```
= CARI KARAKTER =  
1. Cari berdasarkan ID (Interpolation Search)  
2. Cari berdasarkan Nama (Jump Search)  
Pilih metode pencarian: 2  
Masukkan Nama Karakter (Perhatikan Kapital): Ellen  
  
= HASIL PENCARIAN =  
Nama    : Ellen  
Rank    : S  
Role    : Attack  
Elemen  : Ice  
  
Tekan Enter untuk kembali
```

5. Langkah-langkah GIT

5.1 GIT Add

```
C:\Users\Lenovo Gk\OneDrive\Documents\praktikum-apl>git add .  
warning: in the working copy of 'post-test/post-test-apl-7/validasi.h', LF will be replaced by CRLF the next time Git touches it
```

5.2 GIT Commit

```
C:\Users\Lenovo Gk\OneDrive\Documents\praktikum-apl>git commit -m "FINISH PT-7"  
[main 4608928] FINISH PT-7  
3 files changed, 841 insertions(+)  
create mode 100644 post-test/post-test-apl-7/2509106048-Ajis-Aditya-Al-Zahril-PT-7.cpp  
create mode 100644 post-test/post-test-apl-7/2509106048-Ajis-Aditya-Al-Zahril-PT-7.exe  
create mode 100644 post-test/post-test-apl-7/validasi.h
```

5.3 GIT Push

```
C:\Users\Lenovo Gk\OneDrive\Documents\praktikum-apl>git push origin main  
Enumerating objects: 9, done.  
Counting objects: 100% (9/9), done.  
Delta compression using up to 12 threads  
Compressing objects: 100% (7/7), done.  
Writing objects: 100% (7/7), 706.51 KiB | 4.31 MiB/s, done.  
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)  
remote: Resolving deltas: 100% (1/1), completed with 1 local object.  
To https://github.com/VorDecades/praktikum-apl.git  
557c4b2..4608928 main -> main
```